LASM: Extensible Competitive Scheduling Framework for VLIW Assembly Optimization

HONGLI ZHONG, College of Computer Science and Technology, National University of Defense Technology, China and Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, China
ZHONG LIU, College of Computer Science and Technology, National University of Defense Technology, China and Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, China
SHENG MA, College of Computer Science and Technology, National University of Defense Technology, China

Very Long Instruction Word (VLIW) architectures deliver compelling performance-energy-cost advantages for automotive, AI, and high-performance computing. However, a persistent performance-productivity gap limits broader adoption: manual assembly optimization achieves superior performance but requires substantial expertise, while automated compilation underperforms. Existing assembly-level optimization tools commit to single scheduling strategies, unable to adapt to diverse program patterns. This article presents LASM, the first symbolic assembly framework enabling competitive multi-strategy orchestration for complete VLIW programs. Rather than committing to a single algorithm, multiple scheduling strategies simultaneously optimize identical code regions, with the framework automatically selecting the top solution through feasibility-driven ranking. LASM features standardized extension interfaces enabling plugin-style strategy integration and unified scheduling primitives eliminating algorithmic redundancy. We contribute two novel strategies: Longevity-Aware Expanded Modulo Scheduling (LAEMS) with dependency relaxation, proactive reordering, and parallel expansion search, and Front-Back Block Fusion (FBBF) for cross-boundary scheduling. Evaluation on 13 benchmarks across two production VLIW-SIMD processors demonstrates LASM achieves 103.6% and 101.6% of expert assembly performance while reducing manual code by 72.6%. A full YOLOv5 deployment validates production applicability at 95.9% of expert performance. Code is available at https://github.com/urays/lasm.

CCS Concepts: • **Software and its engineering**; • **Computer systems organization → Architectures**;

Additional Key Words and Phrases: VLIW architectures, instruction scheduling, software pipelining, assembly optimization, compiler extensibility, multi-strategy orchestration

ACM Reference Format:
Hongli Zhong, Zhong Liu, and Sheng Ma. 2026. LASM: Extensible Competitive Scheduling Framework for VLIW Assembly Optimization. *ACM Trans. Des. Autom. Electron. Syst.* 1, 1 (July 2026), 42 pages.

1 Introduction

Very Long Instruction Word (VLIW) architectures continue to play a critical role in modern computing, offering an attractive balance of performance, energy efficiency, and silicon cost. By delegating instruction scheduling from hardware to software, VLIW processors achieve superscalar-class performance without complex out-of-order execution logic [1]. This design philosophy proves valuable across diverse domains: Qualcomm's Hexagon Digital Signal Processor (DSP) powers

Authors' Contact Information: Hongli Zhong, urays.cc@gmail.com, College of Computer Science and Technology, National University of Defense Technology, Changsha, China and Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, Changsha, China; Zhong Liu, zhongliu@nudt.edu.cn, College of Computer Science and Technology, National University of Defense Technology, Changsha, China and Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, Changsha, China; Sheng Ma, masheng@nudt.edu.cn, College of Computer Science and Technology, National University of Defense Technology, Changsha, China.

2026. ACM 1557-7309/2026/7-ART
https://doi.org/

billions of smartphones for signal processing [2], Texas Instruments' C7000 family enables real-time automotive safety systems [3, 4], modern AI accelerators incorporate VLIW cores for tensor operations [5, 6], and specialized processors use VLIW designs for High-Performance Computing (HPC) [7].

Despite VLIW architectural advantages, a critical barrier limits broader adoption: developing high-performance VLIW software remains notoriously challenging. Unlike superscalar processors where hardware schedules instructions dynamically, VLIW architectures expose this complexity directly to software. Compilers or programmers must explicitly orchestrate parallel instruction execution, manage resource constraints, and handle complex instruction-level dependencies. These tasks become increasingly complex as modern VLIW processors incorporate Single Instruction Multiple Data (SIMD) vector units, specialized functional units, and custom instruction sets [2, 3, 8]. This creates a persistent **performance-productivity gap**: manual assembly optimization by domain experts delivers substantially better performance by fully exploiting hardware parallelism, but requires scarce expertise and weeks of development time per program. Automated compilation offers productivity but consistently underperforms, with industry benchmarks and academic research documenting significant performance advantages for hand-optimized code [9–12]. Production environments face an uncomfortable choice between economically infeasible manual optimization and automated compilation that leaves substantial performance unrealized.

This performance-productivity gap persists because of a fundamental tension in VLIW compilation: instruction scheduling is computationally intractable [13], yet real-world programs exhibit enormous computational diversity. Different code patterns favor different optimization approaches—aggressive loop pipelining for compute-intensive kernels, efficient list scheduling for independent operations, hot-spot optimization for control-flow-heavy code—but no single scheduling algorithm performs optimally across all patterns. Production compilers address this diversity by committing to fixed scheduling strategies per code pattern [14–17], leaving opportunities unexploited where alternative approaches would excel. Assembly optimization tools similarly employ single algorithms: Texas Instruments' Linear Assembly Optimizer focuses on software pipelining for DSP kernels [18], while CompCert provides formally verified scheduling for complete VLIW programs but commits to a single verified algorithm [19]. Integrating and coordinating multiple scheduling strategies remains challenging across both compiler frameworks and assembly-level optimization tools, as existing infrastructures lack standardized interfaces for strategy composition and mechanisms for automatic candidate selection.

Our Approach. This article introduces LASM (VLIW Assembly Scheduling Machine), the first symbolic assembly scheduling[1] framework enabling competitive multi-strategy orchestration for complete programs beyond isolated kernels. LASM addresses the performance-productivity gap in VLIW development through a dual-community design: *application developers* express programs using symbolic assembly notation that omits tedious scheduling details, while *assembly experts* extend the framework by designing scheduling strategies through unified primitives and standardized interfaces. It orchestrates multiple strategies through competitive evaluation: for each code region, registered strategies simultaneously generate candidate schedules, and the framework selects the

[1]In this article, *symbolic assembly* denotes an abstract input representation in which programmers write raw ISA instructions with symbolic operands, while optionally leaving resource bindings, including functional units and physical registers, unspecified. Scheduling translates this representation into cycle-assigned, bundled execution sequences with fully resolved resource bindings. Relative to fully specified handwritten assembly, this abstraction offers two key advantages. On the one hand, it retains architectural precision, since scheduling strategies can operate with complete knowledge of hardware characteristics, including instruction latencies, functional-unit capabilities, and multi-cycle write-back behavior. On the other hand, it preserves programming flexibility by maintaining full ISA expressiveness without requiring exhaustive manual resource specification. Section 3.2 presents the formal specification of this abstraction.

best solution through two-tier feasibility filtering. This approach combines architectural precision with systematic multi-strategy exploration, substantially reducing development effort while achieving performance competitive with hand-tuned code. Section 2 elaborates the fundamental challenges motivating this design and presents the rationale.

Paper Positioning. This work contributes both an extensible framework and novel scheduling algorithms. The framework provides standardized abstractions (extension interfaces, unified primitives, feasibility-driven selection) enabling fair comparison and composition of diverse strategies. The novel algorithms (LAEMS and FBBF) validate framework expressiveness by achieving expert-competitive performance. This mirrors established precedents: LLVM [14] contributes both its IR and optimization passes; Halide [20] contributes both its scheduling language and concrete schedules. The framework defines the abstraction; the strategies demonstrate its utility.

Key Contributions. Our key contributions include:

- **Extensible Competitive Scheduling Framework.** The first symbolic assembly scheduling framework targeting complete VLIW programs (beyond isolated kernels) and supporting multi-strategy orchestration to compete to schedule identical code regions. Key innovations: standardized extension interfaces for plugin-style strategy integration, unified scheduling primitives eliminating redundancy across implementations, and two-tier feasibility filtering with automatic best-solution selection.
- **Novel Scheduling Strategies.** Two novel strategies demonstrating framework extensibility: *Longevity-Aware Expanded Modulo Scheduling (LAEMS)* with dependency relaxation, proactive store reordering, and parallel expansion search; *Front-Back Block Fusion (FBBF)* for cross-boundary optimization. In our framework, we also implement TDLS [21], IMS [22], and EMS [23] as evaluation baselines.
- **Comprehensive Evaluation.** Evaluation on two production VLIW-SIMD processors with 13 benchmarks achieves 103.6% and 101.6% of expert assembly performance while reducing manual code by 72.6%. YOLOv5 deployment (65 neural network layers) validates production applicability with 95.9% expert performance and 3× productivity improvement.

Paper Organization. Section 2 analyzes fundamental challenges in VLIW instruction scheduling and presents our design principles; Section 3 describes the optimization workflow and foundations; Section 4 presents the competitive scheduling mechanism; Section 5 demonstrates extensible strategy design; Section 6 evaluates performance and productivity; Section 7 discusses related work; Section 8 concludes and outlines future directions.

2 Motivation and Design Rationale

Despite decades of VLIW compilation research, a substantial performance gap persists between compiler-generated code and hand-optimized assembly. This section identifies two fundamental challenges underlying this gap and presents the design principles that guide our approach.

2.1 Challenge 1: Algorithm Diversity Demands Competitive Selection

Observation. The problem of VLIW instruction scheduling is NP-complete [1, 24], precluding any single algorithm from achieving optimal results across all program patterns. This fundamental complexity has driven decades of heuristic development: modulo scheduling variants (e.g., Iterative Modulo Scheduling (IMS) [22], Swing Modulo Scheduling (SMS) [25], Expanded Modulo Scheduling (EMS) [23]) excel at loop pipelining but struggle with irregular control flow, while list scheduling optimizes basic blocks but cannot span boundaries.

Real-world programs exhibit diverse computational patterns, each favoring different optimization strategies. Compute-intensive loops benefit from aggressive software pipelining that maximizes

iteration overlap; control-flow-heavy programs gain more from optimizations targeting frequently executed paths; regions with abundant independent operations favor efficient list scheduling (e.g., Top-Down List Scheduling (TDLS) [21]), whereas those with pipeline bubbles require fusion techniques to fill delay slots. This diversity reflects fundamental trade-offs: strategies optimized for one characteristic often sacrifice performance on others.

Problem. Existing compiler frameworks provide inadequate support for strategy composition, forcing researchers into a binary choice: commit to a single approach or undertake substantial engineering to integrate multiple techniques. For instance, LLVM's MachinePipeliner implements SMS for single-basic-block loops while DFAPacketizer handles bundling through greedy packetization; integrating alternative strategies requires modifying core scheduler classes. These efforts, as community reports indicate, require 6–12 months by experienced compiler specialists [16, 17, 26, 27]. This limitation stems from two technical barriers. First, scheduling strategies tightly embed resource management logic, making composition difficult. Second, frameworks lack mechanisms to automatically evaluate and select among candidate schedules. The cumulative effect is that production compilers typically implement a single, fixed scheduling strategy per code pattern, leaving substantial optimization opportunities unexploited.

Recent research shows that up to 58.38% of efficiency improvements remain achievable beyond standard compiler heuristics [12]. Kalray's work shows that even a heavily customized modern LLVM backend only achieves 93.8% of GCC performance [17], let alone the theoretical optimal performance. These gaps persist precisely because single strategies cannot capture the algorithmic diversity required for optimal scheduling across varied patterns.

Design Principle: Competitive Multi-Strategy Orchestration. We address this challenge through competitive multi-strategy scheduling operating across two orthogonal dimensions. The *first dimension* partitions code regions into specialized scheduling horizons based on structural characteristics. The cyclic horizon applies software pipelining to loops, while the linear horizon compacts basic blocks and fills delay slots. This separation enables multiple strategies to compete within each horizon rather than committing to a single approach. The *second dimension* introduces concurrent strategy competition within each horizon. Multiple strategies simultaneously optimize the same code region, generating diverse candidate schedules that explore different solution space areas. The framework coordinates three key elements: (1) standardized extension interfaces allowing strategies to register at specific horizons without managing the complete pipeline, (2) unified scheduling primitives providing reusable building blocks that eliminate redundant implementations, and (3) two-tier feasibility filtering that validates register allocation constraints before ranking candidates by performance metrics. This design reframes scheduling as systematic exploration where diverse strategies probe different optimization approaches, with the framework automatically selecting the best implementable solution.

2.2 Challenge 2: High Engineering Costs Hinder Productivity

Observation. Application developers face substantial manual optimization costs when targeting VLIW architectures. Achieving peak performance through hand-written assembly demands explicit specification of instruction parallelism, functional unit assignments, register allocation, and delay slot management for every code region. Production deployments indicate that experienced programmers require days to weeks optimizing individual kernels. Hand-tuned assembly binds tightly to specific Instruction Set Architectures (ISAs) and microarchitectures. Retargeting to different processors requires not only rewriting instructions to match new ISAs but also redoing all scheduling decisions, invalidating prior optimization efforts entirely.

Algorithm researchers face different challenges when implementing novel scheduling techniques in existing compiler frameworks. Adapting LLVM for VLIW architectures involves understanding

SSA-based machine Intermediate Representation (IR), modifying scheduler base classes, adapting register allocation mechanisms, and integrating with pipeline managers. Community experience reports provide concrete evidence: the Kalray KV3 backend implementation required managing 570 instructions with multiple variants and extensive modifications to MachinePipeliner, DFAPacketizer, and VLIWPacketizerList [17]. Multiple independent reports consistently indicate 6–12 months of focused development time for experienced compiler specialists to implement production-quality VLIW backends [16, 17, 28]. For academic researchers without prior LLVM experience, these timelines can extend to years.

Problem. These engineering barriers create a cycle limiting VLIW adoption. High development costs slow research iteration cycles, discouraging exploration of novel scheduling techniques. In industrial deployment, manual optimization remains the only viable path to peak performance, but the required expertise is scarce and expensive. More fundamentally, tight coupling between programs and specific processor implementations prevents software reuse, creating vendor lock-in that discourages architectural experimentation.

Design Principle: Dual-Layer Cost Reduction. Our framework reduces engineering costs for both communities through targeted abstractions at each level. For application developers, LASM introduces VLIW Assembly Notation (LAN), a symbolic assembly abstraction that lets programmers omit tedious low-level specifications: instruction parallelism, functional unit assignment, register allocation, and delay slot management. When retargeting to different processors, developers update instruction mnemonics to match new ISAs while the framework automatically handles scheduling through declarative machine model abstractions. This mechanical transformation proves far less expensive than complete manual reoptimization. The internal representation (Stratified Assembly Structure, SAS) supports mixed scheduling states where scheduled and unscheduled code coexist, enabling iterative refinement workflows where manual expertise and automated scheduling collaborate progressively.

For algorithm researchers, the framework employs a dual-layer extensibility mechanism. Low-level unified scheduling primitives encapsulate complex dependency analysis and resource checking as reusable components, eliminating algorithmic redundancy across strategy implementations. Standardized interfaces enable plugin-style integration requiring minimal coding effort. This separation allows researchers to focus on encoding their optimization insights as strategies without managing the entire optimization workflow.

2.3 Design Rationale: Symbolic Assembly Framework

The design principles above motivate our choice of a symbolic assembly framework rather than extending high-level compiler backends. This positioning provides three strategic advantages that directly enable the competitive multi-strategy mechanism and dual-layer cost reduction.

First, *complete hardware visibility*: Symbolic assembly maintains one-to-one correspondence with target ISA instructions while preserving precise knowledge of instruction latencies, functional unit assignments, and multi-cycle write-back sequences. This visibility is essential for accurate resource conflict detection and feasibility validation during competitive selection. Second, *flexible register coordination*: Symbolic assembly supports resource management strategies spanning from fully manual to fully automated. Users may explicitly assign registers for performance-critical regions while delegating routine allocations to the framework, or provide entirely symbolic code for full automation. This flexibility enables progressive optimization workflows where expert register assignments guide scheduling decisions in hot paths, while automated allocation handles less critical regions. During competitive candidate generation, feasibility filtering also validates register pressure before selection, rejecting over-constrained schedules early. This coordination between scheduling and allocation reduces the phase-ordering challenges that plague compiler

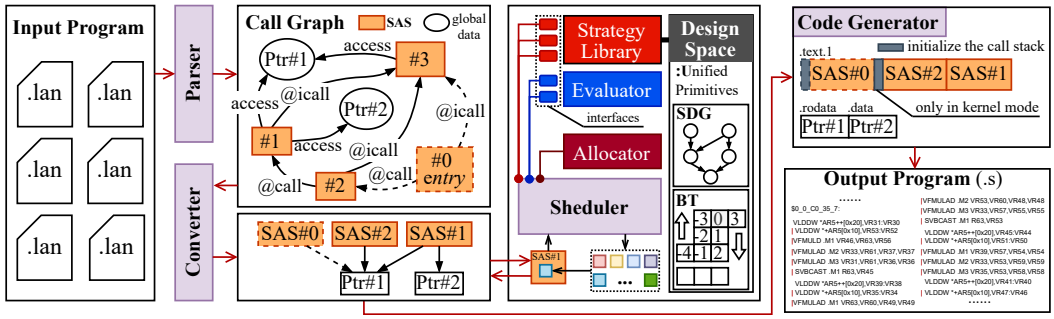


Fig. 1. LASM optimization workflow. The input is a complete program or function library distributed across multiple LAN files; the output is optimized assembly code. The process consists of four stages: (1) The **parser** analyzes input files and constructs a call graph linking all functions (stored as SAS) and global data. (2) The **converter** simplifies the call graph, expands pseudo-operations (e.g., converting `@call` into push/jump/pop sequences), performs control flow refinement, and generates a scheduling queue of SAS functions. (3) The **scheduler** processes each SAS block-by-block, invoking registered strategies to generate candidate schedules that compete through feasibility filtering and performance evaluation. (4) The **code generator** encodes all SAS instructions, optimizes memory layout, and produces the final assembly output.

backends [29]. Third, *mixed scheduling state support*: Symbolic assembly naturally accommodates the coexistence of fully scheduled code (with explicit resource bindings) and symbolic unscheduled code (with variables and abstract ordering) within a unified representation. Traditional compiler IRs maintain uniform abstraction levels, forcing rigid pipelines. They are incompatible with progressive multi-horizon scheduling where cyclic strategies produce fully scheduled kernels while linear strategies process surrounding regions that remain symbolic.

3 System Overview and Representations

This section outlines the LASM framework. Figure 1 illustrates its compilation workflow that processes LAN programs through four stages: parsing and call graph construction, pseudo-operation expansion and scheduling queue generation, multi-strategy competitive scheduling, and final code generation with memory layout optimization.

The framework comprises three foundational components. Machine Model Abstraction (Section 3.1) provides declarative processor specifications through template-based rules, decoupling optimization algorithms from architecture-specific details. VLIW Assembly Notation (Section 3.2) allows developers to write actual VLIW instructions while omitting tedious low-level specifications. Stratified Assembly Structure (Section 3.3) serves as the persistent internal structure uniquely supporting mixed scheduling states where scheduled and unscheduled code coexist. These components work together to support the extensible competitive scheduling mechanism (Section 4) and extensible strategy design (Section 5).

3.1 Machine Model Abstraction

The machine model provides declarative processor specifications through template-based rules, decoupling scheduling algorithms from architecture-specific details. Instructions are treated as opaque scheduling units characterized solely by resource consumption and operand timing, enabling strategy reuse across processors through model updates without modifying scheduling logic.

The model specifies three resource categories. **Functional units** receive fine-grained integer identifiers enabling precise modeling of heterogeneous units—critical for processors like Matrix

DSP [8] with subtle latency differences across functional units. Unlike traditional backends tracking only unit counts per type, LASM's fine-grained identification enables accurate conflict detection during resource validation.

Register banks are declared through ternary descriptors `(P,X,Y)` where prefix `P` identifies the bank and intervals `X`, `Y` define the number range in two dimensions. This naturally captures both simple scalar organizations and complex structures including overlapping physical storage[2]. The descriptor enables automatic type inference through interval semantics, where registers sharing the same prefix but differing in numbering dimensions indicate overlapping physical storage, facilitating automatic dependency inference.

This abstraction supports multi-bank heterogeneous organizations where the interference graph naturally partitions by register prefix (variables in different banks occupy disjoint physical storage and do not conflict). Inter-bank data movement (e.g., scalar-to-vector transfers) is modeled through dedicated copy instructions with specified latencies. The current implementation targets shared-access architectures where all functional units access all banks with uniform latency. For clustered architectures requiring non-uniform communication modeling, the framework's plugin architecture accommodates cluster-aware scheduling strategies as extensions.

Instruction templates specify assignable functional units, operand access patterns (registers, immediate constants, or memory addresses), bit-width ranges for constants, and precise latency information including when each operand occupies ports for reading, writing, and write-back completion. This latency specification supports modern VLIW features including multi-cycle write-back sequences where a single instruction may occupy write ports across multiple cycles. The model accommodates specialized architectural features through explicit markers for cooperative operations (requiring execution of multiple machine instructions) and barrier instructions (enforcing execution isolation).

3.2 VLIW Assembly Notation

LASM operates at the symbolic assembly level, an abstraction positioned between high-level compiler IRs and fully scheduled machine code. At this level, programmers write instruction mnemonics (with symbolic variables) from the target VLIW ISA, optionally omitting low-level scheduling details: (1) instruction bundling for parallel execution, (2) functional unit assignments, (3) physical register allocation, and (4) delay slot management.

Based on this abstraction, we introduce VLIW Assembly Notation (LAN), a new design for assembly-level program representation. Variables are implicitly declared on first use and remain symbolic throughout the user's code. Importantly, LAN supports selective specification. Users may provide these details partially (e.g., bundling critical instructions while leaving others unspecified) or completely, and the framework respects all user-provided specifications while automatically handling omitted details.

Pseudo-operations marked with `@` provide structured programming capabilities: `@func` and `@main` define regular functions and program entry points; `@import` includes external functions enabling modular organization; `@icall` requests function inlining; `@ret` marks function returns; `@call` performs standard function invocation. Additional pseudo-operations support immediate value specification (e.g., `@b64`, `@b36`, `@bf`), multi-core synchronization (`@bar`), and loop bound annotations for optimization hinting.

Unlike traditional assembly and linear assembly [18], LAN enables decomposing programs into multiple functions and files for improved code readability and reuse. To minimize calling

[2]In Matrix DSP [8], the Shared Vector Register (SVR) is a vector register spanning 16 processing units. In addition to vector access, it supports scalar access as 16 separate scalar registers, namely SVR0–SVR15.

```
1  @sect ".text.1"
2  @func _axpby_kernel addrX, addrY, N, alpha, beta {
3      SMVAGA     addrX, ar0
4      SMVAGA     addrY, ar1
5   |  SMVAGA     addrY, ar2 ;Specify to be issued
           simultaneously with the previous instruction.
6      SAND   .L   0x1F, N, c ;Specify a functional unit.

7      SSHFLR     5, N, i
8  [c] SADD       1, i, i
9  LOOP_CORE:4096,65536 ;Specify 4096<=N<=65536
10     VLDDW     *ar1++[16], y16_1:y16_0
11     VLDDW     *ar0++[16], x16_1:x16_0
12     SVBCAST   alpha, v_alpha
13     SVBCAST   beta, v_beta
14     VFMULD    x16_1, v_alpha, x16_1_r
15     VFMULD    x16_0, v_alpha, x16_0_r
16     VFMULAD   y16_1, v_beta, x16_1_r, x16_1_r
17     VFMULAD   x16_1, v_beta, x16_0_r, x16_0_r
18     VSTDW     x16_1_r:x16_0_r, *ar2++[16]
19 [i] SSUBU     1, i, i
20 [i] SBR       LOOP_CORE
21     @ret i, ar2 ; Only for testing
22 }
```

```
1  @import "./MT3000-GPDSP/lib/csl.lan", test_output,
       test_stop, config_cvccr, config_scr
2  @import "./MT3000-GPDSP/kernels/axpby.lan",
       _axpby_kernel
3
4  @sect ".text.s"
5  @main testing_axpby { ; Entry function.
6      @b36 addrX, 0x210000000
7      @b36 addrY, 0x210060000
8      @b64 N, 32768
9      @b64 alpha, 0x404c000000000000
10     @b64 beta, 0xc040000000000000
11     @icall config_cvccr, 3
12     @icall config_scr, 3
13     @bar ;In single-core mode, used only for marking
            the address of the next instruction.
14     @icall _axpby_kernel, addrX, addrY, N, alpha,
           beta,=r1,r2 ;Here, we omit the checks for
           i and ar2.
15     @bar
16 END: ;Only used to improve program readability.
17     @icall test_output, addrY, N*8 ; N x double
18     @icall test_stop
19 }
```

Fig. 2. A LAN program implementing $\alpha X + \beta Y$ vector operation, demonstrating modular composition across three files. **Left:** The computational kernel `_axpby_kernel` (`axpby.lan`) illustrates LAN's selective specification—explicit instruction parallelism (line 5, marked `|`), functional unit assignment (line 6, `.L`), and loop bounds (line 9)—while omitting other low-level details to delegate scheduling to LASM. **Right:** The entry function `testing_axpby` (`testing_axpby.lan`) demonstrates complete program structure with modular imports (`@import`, lines 1–2), bit-width-specific parameter initialization (`@b36`, `@b64`, lines 6–10), selective return value capture (line 14, `=r1,r2`), and function calling (`@icall`, lines 11–12,14,17–18).

overhead, LASM applies techniques such as function inlining (guided by `@icall` hints) and tail-call elimination [30]. Figure 2 illustrates a complete LAN program implementing αX + βY operation on vectors of length N. When retargeting to different processors, developers update instruction mnemonics to match new ISAs (a mechanical transformation for assembly programmers) while keeping variables and resource specifications symbolic; the framework handles scheduling through updated machine models. This substantially reduces porting effort compared to complete manual reoptimization, as our dual-processor experiments demonstrate (Section 6).

3.3 Stratified Assembly Structure

The Stratified Assembly Structure (SAS) provides the persistent internal representation that distinguishes LASM from conventional compiler infrastructures. Its defining capability is supporting mixed scheduling states, where fully scheduled and symbolic unscheduled code coexist within a single function. Traditional compiler IRs enforce uniform abstraction—either machine-level code with explicit cycle assignments or symbolic operations with virtual registers—necessitating rigid pipelines that transform entire functions in monolithic phases. SAS relaxes this constraint to accommodate two practical workflows: (1) incremental human-automated collaboration, where developers hand-optimize performance-critical regions while delegating remaining code to automated strategies, and (2) multi-horizon progressive optimization, where cyclic strategies generate scheduled loop kernels while linear strategies subsequently process surrounding regions that remain symbolic.

SAS represents code in two block categories reflecting distinct optimization states. **Symbolic basic blocks** preserve the assembly-level abstraction: instructions reference virtual variables, target abstract functional units, and carry no cycle assignments. **Scheduled beat blocks** encode fully realized execution sequences: each beat specifies a parallel instruction bundle with concrete

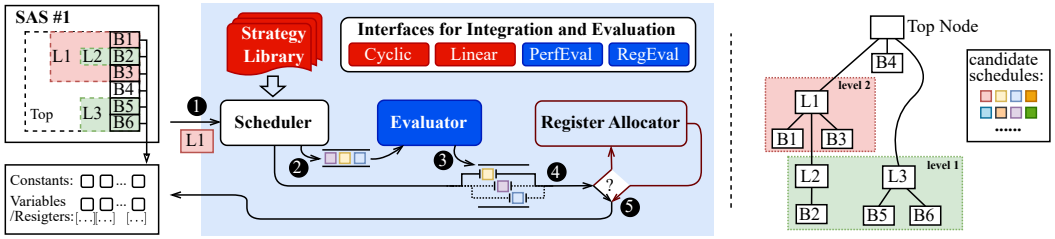


Fig. 3. Overview of the scheduling mechanism (highlighted by blue background). Here, B1~B6 are basic blocks, and L1~L3 are loop blocks. Using loop block L1 as an example: ❶ The **scheduler** first extracts L1 from the target SAS; ❷ it then invokes the **strategy library** to generate candidate schedules. ❸ These schedules are scored by the evaluator, and those with the highest ranking are selected first. ❹ Users can configure the default incremental **register allocator** to be enabled immediately for the schedule, or they can choose to skip it. Eventually, ❺ the selected schedule is formally submitted to the target SAS to replace the original L1. To the right of the dashed vertical line: A schedule tree corresponding to the target SAS. Each square node has at least one schedule associated with all its child nodes. The scheduling order proceeds from largest to smallest according to level number. Nodes at the same level can be scheduled in parallel.

functional unit bindings and physical register allocations, capturing all timing constraints including multi-cycle write-back. The core scheduling task—determining execution timing, instruction grouping, and resource bindings subject to dependency constraints—transforms symbolic blocks into scheduled beats. This transformation proceeds incrementally; within any SAS function, scheduled and symbolic regions may coexist.

Each LAN function (declared via `@func` or `@main`) corresponds to one SAS instance comprising two components: a data manager aggregating constants and variables with their allocation constraints, and a block tree organizing instructions and control flow. The block tree is a hierarchical structure with four node types: root blocks serving as function-level containers, loop blocks capturing iterative constructs with arbitrary nesting, basic blocks holding straight-line instruction sequences, and label blocks marking branch targets. Each node maintains a parent pointer and provides $O(1)$ access to its children, enabling efficient predecessor and successor traversal. Multiple SAS instances connect through a call graph constructed by analyzing function invocations and global variable accesses.

Within the block tree, LASM partitions code into basic blocks at control transfer instructions (branches, jumps, returns) and user-defined labels. Block boundaries remain fixed throughout analysis; however, strategies can reason about cross-boundary optimization via a dependence graph (Section 5.1.1). Beyond intra-block data dependencies, the graph annotates cross-block value flow through specialized branch-related edge types, furnishing the unified dependency view that scheduling strategies require for exploiting instruction-level parallelism across block boundaries.

Prior to scheduling, the LASM converter applies code refinements to establish a clean baseline: dead code elimination discards instructions with unused results, loop-invariant code motion relocates redundant computations outside loops, and function inlining eliminates call overhead for small routines. These preparatory transformations yield an optimized SAS in which scheduling strategies can focus on instruction-level parallelism. The mixed-state design, combined with this refined input, enables the scheduling mechanism (Section 4) to operate seamlessly across optimization boundaries.

4 Extensible Competitive Scheduling Mechanism

This section presents the extensible competitive scheduling mechanism, which addresses Challenge 1 from Section 2. Rather than committing to a single scheduling approach, the framework orchestrates multiple strategies that compete across two orthogonal dimensions: different scheduling horizons targeting distinct code region types, and concurrent strategies within each horizon that simultaneously explore alternative schedules for the same region.

Figure 3 illustrates the complete workflow. The scheduler iteratively processes code regions, invoking registered strategies to generate competing candidates that undergo feasibility-driven selection. Section 4.1 introduces the two-horizon strategy registration that enables plugin-style strategy integration. Section 4.2 details the competitive selection through two-tier feasibility filtering.

4.1 Strategy Registration and Orchestration

LASM defines two standardized interfaces corresponding to distinct scheduling horizons in the optimization pipeline. Each horizon targets a specific category of code regions, reflecting the principle that different structural patterns demand specialized scheduling techniques.

The **cyclic horizon** addresses loop blocks through software pipelining techniques designed to maximize iteration-level parallelism. In contrast, the **linear horizon** processes basic blocks containing straight-line code, emphasizing instruction bundling and delay slot filling within acyclic control flow. The registration interfaces are defined as follows:

```
// The interface of cyclic scheduling for SAS loop blocks.
using CyclicScheduleInterface = void(*)(const LoopBlock&, const MetaManager&, const Option&, Schedules&);
// The interface of linear scheduling for SAS basic blocks (or beat blocks).
using LinearScheduleInterface = void(*)(const BasicBlock&, const MetaManager&, const Option&, Schedules&);
```

Each interface accepts four parameters with consistent semantics across horizons. The first parameter specifies the target code region: `LoopBlock` for cyclic scheduling or `BasicBlock` for linear scheduling. The second parameter, `MetaManager`, provides read-only access to variable metadata and constant information required for dependency analysis. The third parameter, `Option`, supplies user-specified directives such as unrolling factors or pipelining budgets. The fourth parameter holds a set of candidate schedules generated by the strategy.

4.1.1 Multi-Round Progressive Scheduling. The scheduler employs a multi-round approach for progressive instruction scheduling: *Cyclic horizon.* Iteratively traverses the SAS from innermost to outermost loops, grouping unmarked loop blocks by nesting level (Figure 3). Same-level blocks are processed in parallel, with each thread applying registered cyclic strategies. Strategies may accept or decline blocks based on pattern matching and can generate new loop blocks (e.g., remainders or transformed nests), which are scheduled in subsequent rounds. A block is marked only after all strategies have considered it, ensuring one-time processing. Terminates when no unmarked loop blocks remain. *Linear horizon.* Executes after all loops are scheduled. In each round, selects one unmarked basic block and runs all registered linear strategies in parallel. Strategies access neighboring regions through the selected block, enabling multi-block scheduling per round. Processed blocks are marked to prevent reprocessing.

4.1.2 Strategy Composition. This design enables two forms of strategy composition. *Cross-horizon composition* allows different strategies to specialize at different scheduling levels. For instance, modulo scheduling variants at the cyclic horizon and list scheduling or fusion techniques at the linear horizon. *Intra-horizon competition* permits multiple strategies to simultaneously target the same horizon, each exploring different solution space regions. For example, multiple modulo

scheduling variants with different initiation interval selection heuristics can compete at the cyclic horizon, while multiple linear scheduling strategies compete at the linear horizon.

Strategies operate independently without requiring knowledge of other registered plugins. Each strategy receives a code region, applies its scheduling logic, and returns candidate schedules. The framework manages all orchestration: strategy invocation, candidate collection, and competitive selection.

4.2 Feasibility-Driven Competitive Selection

The competitive mechanism frames scheduling as a solution space exploration problem: diverse strategies probe distinct regions of the optimization landscape, while the framework automatically identifies the best implementable solution. Traditional schedulers often discover resource violations only after extensive scheduling effort, forcing backtracking or emergency spilling that degrades performance. LASM inverts this paradigm through a two-stage feasibility filtering process: performance-oriented metrics first establish a ranked candidate ordering, followed by resource constraint validation via register allocation. This early rejection prevents wasted computation on infeasible solutions while enabling strategies to explore aggressive scheduling transformations.

4.2.1 Two-Tier Evaluation Metrics. The evaluation mechanism employs two customizable comparison functions that establish a ranked ordering over candidate schedules. The first-tier `PerfEval` metric assesses scheduling quality:

```
const auto PerfEval = [&](const BlockInfo& obj, const Schedule& lhs, const Schedule& rhs) ->bool {
  // Prefer the schedule that reduces more cycles compared to the unscheduled baseline.
  return Eval::cycle_reduction(lhs) > Eval::cycle_reduction(rhs); // lhs wins.
};
```

The built-in function `Eval::cycle_reduction()` estimates the number of execution cycles reduced by a schedule relative to the unscheduled target code region, in which LASM automatically accounts for required delays with nops for each SAS instruction. VLIW instruction scheduling ensures compile-time arrangement determines runtime behavior. For runtime-variable parameters like loop-counting variables, LASM uses LAN syntax to guide optimization decisions. As shown in Figure 2 line 9, `LOOP_CORE:4096,65536` specifies that the initial value of `i` (line 19) satisfies $4096 \leq i \leq 65536$. This information enables strategies to be optimized for expected execution patterns rather than worst-case or arbitrary assumptions.

Common `PerfEval` implementations optimize for execution latency (minimizing cycle count), code size (minimizing instruction bytes), or weighted combinations thereof. The `BlockInfo` parameter provides block metadata, enabling users to implement context-sensitive evaluation strategies. For example, users could prioritize low-latency schedules for loop blocks while favoring compact code for basic blocks.

The second-tier `RegEval` metric refines ranking among performance-equivalent schedules by prioritizing solutions with lower register requirements:

```
const auto RegEval = [&](const BlockInfo& obj, const Schedule& lhs, const Schedule& rhs) ->bool {
  // Prefer the schedule with lower register pressure.
  return Eval::register_pressure(lhs) < Eval::register_pressure(rhs); //lhs wins.
};
```

The built-in function `Eval::register_pressure()` estimates register demand by computing the maximum number of variables simultaneously live at any program point. Specifically, the function constructs a liveness profile across the schedule, identifying for each execution cycle the set of variables with overlapping live ranges, and returns the peak occupancy observed. This

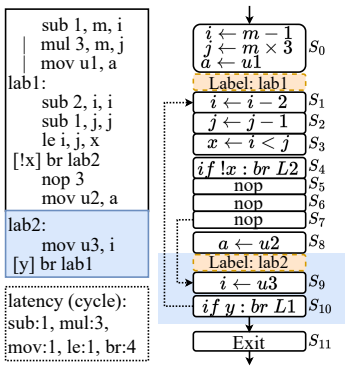


Fig. 4. Schematic of an initial VST showing control flow structure across mixed scheduled and unscheduled code regions. The region above label "lab2" contains scheduled instructions with explicit cycle assignments, while the region below remains unscheduled.

Algorithm 1: VST Completion

Input: Initial VST $\mathcal{S}$ (after control flow structure)
Output: Complete VST $\mathcal{S}$ with variable sets
Notation:
I_{use}, I_{def}: Variables read/written by instruction I
λ_I^a: Write-back delay (cycles) for variable a in I
ω_I^a: Cycle offsets when I occupies write ports for a
ISCV(X): True if variable X is loop-carried
ISPMI(I): True if I is predicated or last in cooperative op
SKIP(I, a): True if a should skip DU update
$|\mathcal{S}|$: Total number of state nodes in VST

// Phase 1: Initialize U, DU based on instruction effects
01 : **for** $I \in Inst(\mathcal{S}[t]), t \in [0, |\mathcal{S}|)$ **do**
02 : $\quad \mathtt{U}(\mathcal{S}[t]) \leftarrow \mathtt{U}(\mathcal{S}[t]) \cup I_{use}$
03 : $\quad$ **for** $\tau \in \{\lambda_I^a \mid a \in I_{def}, \neg\text{SKIP}(I, a)\}$ **do**
04 : $\quad\quad$ Add a to $\mathtt{DU}(\mathcal{S}[t + \tau - 1])$
// Phase 2: Complete IN, OUT via reverse dataflow
05 : **while** $\exists t : \mathtt{IN}(\mathcal{S}[t])$ updated last round **do**
06 : $\quad$ **for** $t = |\mathcal{S}| - 1$ **down to** 0 **do**
07 : $\quad\quad \mathtt{OUT}(\mathcal{S}[t]) \leftarrow \bigcup_{n \in Succ(\mathcal{S}[t])} \mathtt{IN}(n)$
08 : $\quad\quad \mathtt{IN}(\mathcal{S}[t]) \leftarrow \mathtt{U}(\mathcal{S}[t]) \cup (\mathtt{OUT}(\mathcal{S}[t]) \setminus \mathtt{DU}(\mathcal{S}[t]))$
// Phase 3: Complete DW based on write port occupancy
09 : **for** $I \in Inst(\mathcal{S}[t]), t \in [0, |\mathcal{S}|)$ **do**
10 : $\quad$ **for** $\tau \in \omega_I^a, \forall a \in I_{def}$ **do**
11 : $\quad\quad$ Add a to $\mathtt{DW}(\mathcal{S}[t + \tau])$

estimation occurs during competitive candidate generation, providing early filtering before the actual register allocation phase that follows schedule selection.

This two-tier design reflects the practical optimization principle: among schedules achieving similar performance, those requiring fewer registers are more likely to succeed in allocation validation and leave more registers available for surrounding code regions. The evaluator applies `PerfEval` as the primary sort key with `RegEval` serving as the tiebreaker, yielding a ranked candidate list where top entries represent optimal performance-resource trade-offs.

4.2.2 Unified Liveness Analysis for Mixed Code States. A key difficulty in feasibility validation stems from the progressive scheduling model of LASM. At any intermediate compilation stage, an SAS function may include both scheduled code regions, in which instruction latencies and beat-block structure are explicit, and unscheduled code regions, in which instructions remain organized as basic blocks with symbolic ordering. This hybrid representation is not directly compatible with conventional liveness analysis, which generally assumes a single, uniform program representation. In addition, VLIW latency semantics further complicate the analysis. If a variable is defined at cycle t by an instruction with latency 5, the value becomes available only at cycle $t + 5$, and the associated write port may remain occupied during the intervening cycles.

We address this issue with the Variable State Timeline (VST), a temporary intermediate structure derived from the SAS for liveness analysis. The VST flattens the hierarchical SAS representation into a linear sequence of state nodes, where each node corresponds to one execution cycle. Each state node records five sets: `U` for variables read in the cycle, `DU` for newly defined variables, `IN` for variables live at entry, `OUT` for variables live at exit, and `DW` for variables whose defining instructions still occupy write ports. For scheduled beat blocks, a state node corresponds to one explicitly scheduled cycle with bundled instructions. For unscheduled basic blocks, each instruction is modeled as taking one cycle.

Formally, let $\mathcal{S}$ denote a VST consisting of a sequence of state nodes indexed by execution cycles. For cycle $t \in [0, |\mathcal{S}|)$, we use $\mathcal{S}[t]$ to denote the state node at cycle t, and $Inst(\mathcal{S}[t])$ to denote the set of SAS instructions beginning execution at cycle t. Each state node $\mathcal{S}[t]$ maintains the five variable sets described above and a successor set $Succ(\mathcal{S}[t])$ representing possible next execution

points. Control flow structure determines each state node's successors based on block type and branching behavior. For straight-line code within a basic block, $Succ(\mathcal{S}[t]) = \{\mathcal{S}[t+1]\}$. For branch instructions that transfer control between blocks, $Succ(\mathcal{S}[t])$ includes both the branch target state node and the fall-through successor. Loop blocks with back-edges create cyclic VST structures where $Succ(\mathcal{S}[t])$ may point to earlier state nodes, i.e., $Succ(\mathcal{S}[t]) \ni \mathcal{S}[t']$ where $t' < t$.

The VST construction from a given SAS code region proceeds in two steps: First, control flow structure is established by traversing the SAS block hierarchy and analyzing branch instructions to identify their trigger cycles, producing an initial VST as shown in Figure 4. Scheduling within specific horizons avoids overlapping delay slots between code regions scheduled in different phases, allowing jump trigger moments to be determined locally. The block structure in the SAS explicitly marks boundaries between scheduled and unscheduled regions, enabling the VST to apply different latency models accordingly.

Second, Algorithm 1 computes the five variable sets for each state node through three phases: initializing U and DU based on instruction effects, computing IN and OUT sets through reverse dataflow analysis, and calculating DW sets based on write port occupancy. The algorithm handles architectural features through specialized predicates. The SKIP(I, a) predicate identifies two categories requiring special treatment: (1) loop-carried variables (detected via ISCV(a)) that maintain values across loop iterations in the SAS, and (2) predicated or cooperative operations where only specific instructions in a sequence update the destination register (detected via ISPMI(I) combined with checking if a was already defined earlier). For predicated instructions, the analysis conservatively extends liveness to account for conditional execution, while cooperative operations track only the final instruction's write effects. These considerations ensure correct liveness analysis for predicated execution, multi-cycle cooperative operations, and loop-carried dependencies—all architectural features supported by the SAS representation.

Once the VST is constructed, building the interference graph becomes straightforward. Two variables interfere if simultaneously live or simultaneously requiring write ports:

$$\bigcup_{t \in [0, |\mathcal{S}|)} [\mathtt{OUT}(\mathcal{S}[t]) \times \mathtt{DU}(\mathcal{S}[t])] \cup [\mathtt{DU}(\mathcal{S}[t]) \times \mathtt{DU}(\mathcal{S}[t])] \cup [\mathtt{DW}(\mathcal{S}[t]) \times \mathtt{DW}(\mathcal{S}[t])] \tag{1}$$

where $A \times B$ denotes the set of interference edges $\{(x, y) \mid x \in A, y \in B, x \neq y\}$. This interference graph serves as the foundation for subsequent register allocation. After allocation completes, the VST is discarded—it exists only as a temporary analysis structure, while the persistent program representation remains in the SAS.

4.2.3 Adaptive Register Allocation with Sequential Fallback. The register allocation mechanism in LASM serves the scheduling framework; the novelty lies in the VST's ability to perform unified liveness analysis across mixed scheduling states (Section 4.2), not in the allocation algorithms themselves, which follow standard graph-coloring techniques. We clarify that the terms "allocation during scheduling" and "allocation after scheduling" used in this section describe *allocation timing and scope* rather than distinct algorithmic approaches: the former refers to strategies incrementally assigning registers while bundling instructions within individual regions, whereas the latter refers to post-scheduling allocation that achieves global resource visibility across all scheduled regions. All workflows employ identical VST-based interference graph construction and standard graph-coloring; the distinction lies in when and where allocation decisions are made, not in the coloring algorithm itself.

Allocation timing flexibility. Register allocation in VLIW scheduling faces a fundamental tension between local and global visibility. Allocating during scheduling enables informed bundling decisions but restricts the allocator to individual regions. Conversely, post-scheduling allocation

achieves global resource management but cannot influence scheduling choices. LASM accommodates both through flexible allocation integration.

The framework supports three workflows: allocation during scheduling (where strategies incrementally assign registers while bundling instructions), allocation after scheduling (where strategies generate symbolic schedules for subsequent global allocation), and hybrid allocation (where strategies explicitly allocate critical variables while delegating remaining variables to the default allocator). When strategies perform custom allocation, the framework respects those decisions and only processes unassigned variables.

The progressive scheduling pipeline provides an opportunity for strategic allocation ordering. Since innermost loops are scheduled first, variables predominantly used within these performance-critical regions can be prioritized for physical register allocation, allowing spilling decisions to favor less frequently executed outer regions.

Sequential fallback for the spill-compaction trade-off. The built-in allocator uses the VST-derived interference graph with standard graph-coloring. The key technical contribution lies in its *sequential fallback mechanism*, which directly addresses the fundamental trade-off between instruction compaction and register pressure. Aggressive schedules that achieve tighter instruction compaction typically increase register pressure by overlapping more variable lifetimes; conversely, conservative schedules with lower register requirements may sacrifice instruction-level parallelism.

Rather than selecting only the top-ranked candidate, the allocator processes sorted candidates sequentially until one succeeds. This design operationalizes the spill-compaction trade-off: candidates achieving aggressive compaction but higher register pressure are attempted first; if allocation fails due to insufficient registers, the system gracefully degrades to less compact but allocatable alternatives. This approach is superior to immediate spilling because it preserves parallelism within allocatable schedules while avoiding the performance penalty of additional load/store operations.

Sequential fallback achieves robustness without sacrificing optimization quality. If the top-ranked schedule fails allocation, the system automatically attempts the next-best alternative rather than forcing expensive backtracking or resorting to spilling. In practice, the mechanism typically terminates within the first 15–20 candidates, as `RegEval` effectively pre-filters by resource feasibility. When allocation is disabled (user-controlled via configuration), the top-ranked schedule proceeds directly to submission, supporting workflows where allocation is handled externally or deferred to later compilation phases.

Spilling as last resort. When no candidate schedule can be allocated successfully, spilling is invoked as a final fallback. Spill candidates are determined by a cost model that prioritizes variables with long live ranges, low access frequency, and limited participation in performance-critical loops. For each selected variable, spill code is introduced by inserting a store after each definition and a load before each use. Accordingly, the allocation procedure first attempts sequential fallback and resorts to spilling only when necessary, thereby favoring spill-free solutions whenever available. Across the 26 benchmark instances evaluated on two VLIW platforms, the `RegEval` tie-breaker is sufficient to guide selection to allocatable schedules, and no spilling occurs.

The complete allocation workflow can thus be viewed as a three-stage hierarchy. First, `RegEval` pre-filtering orders candidate schedules by estimated register pressure, promoting schedules with a higher likelihood of successful allocation. Second, sequential fallback traverses this ordered set until an allocatable schedule is identified, balancing schedule compactness against allocation feasibility. Third, spilling is applied only if all candidate schedules remain unallocatable, thereby preserving compilation robustness while limiting performance loss through cost-aware spill selection.

5 Unified Primitives and Novel Scheduling Strategies

The competitive scheduling mechanism establishes standardized extension interfaces for strategy registration and evaluation. Building upon this foundation, this section introduces unified scheduling primitives (Section 5.1) and demonstrates rapid strategy prototyping through two novel contributions: Longevity-Aware Expanded Modulo Scheduling (LAEMS, Section 5.2) and Front-Back Block Fusion (FBBF, Section 5.3).

5.1 Unified Scheduling Primitives

To reduce algorithmic redundancy across strategies, we introduce three scheduling primitives built upon two data structures that encapsulate complex operations as reusable building blocks.

5.1.1 Scheduling Dependence Graph. The Scheduling Dependence Graph (SDG) captures fine-grained instruction dependencies within VLIW code regions, combining control flow and data flow into a unified representation. Unlike traditional dependence graphs modeling only inter-instruction relationships, the SDG explicitly represents implicit dependencies at control flow boundaries through specialized edge types. We define the SDG as a directed graph $G = (V, E)$ where each vertex $v \in V$ represents a SAS instruction (may correspond to multiple machine instructions). Each directed edge $e^{\alpha}_{u,v} \in E$ from vertex u to vertex v encodes a dependency caused by trigger variable α. Dependencies are classified into five types: $\delta(e) \in \{\text{RAW}, \text{WAR}, \text{WAW}, \text{AJ}, \text{BJ}\}$, where Read-After-Write (RAW), Write-After-Read (WAR), and Write-After-Write (WAW) represent classical data dependencies.

The AJ (After-Jump) and BJ (Before-Jump) types capture implicit dependencies at control flow boundaries. For a branch operation b, an operation u preceding b has an AJ dependency to b if u performs a store operation, modifies branch-condition variables, or updates variables that remain live after b. Conversely, an operation w in a target block has a BJ dependency from b if w reads variables potentially modified before b, writes to memory, or performs barrier operations. These implicit dependencies ensure correct instruction bundling across control flow transitions.

Each edge $e^{\alpha}_{u,v} \in E$ maintains three attributes:

- $\lambda(e) = (\lambda_u, \lambda_v)$: Latency cycles required for operations u and v to process variable α. For a RAW dependency where u writes α in 3 cycles and v reads α immediately, $\lambda(e) = (3, 0)$.
- $\omega(e) = (\Omega_u, \Omega_v)$: Sets of cycle offsets when operations u and v occupy functional unit write ports. Modern VLIW architectures support multi-cycle write-back [7, 8]. For an instruction writing to a register pair requiring two consecutive write-backs starting at relative cycle 1, $\Omega_u = \{1, 2\}$.
- $\rho(e)$: Iteration distance for loop-carried dependencies. If $\rho(e) = k > 0$, then operation v in iteration i depends on operation u in iteration $i - k$. For dependencies within the same iteration or in acyclic regions, $\rho(e) = 0$.

Each vertex $v \in V$ maintains two computed attributes derived from graph topology:

- $\text{depth}(v)$: Maximum latency-weighted distance from any source vertex to v, considering only intra-iteration dependencies ($\rho = 0$):

$$\text{depth}(v) = \max_{u \in \text{pred}(v)} \left(\text{depth}(u) + \lambda_u(e^{\alpha}_{u,v})\right) \tag{2}$$

  where $\text{pred}(v) = \{u \mid e^{\alpha}_{u,v} \in E\}$ and $\text{depth}(v) = 0$ for source vertices.
- $\text{height}(v)$: Maximum latency-weighted distance from v to any sink vertex:

$$\text{height}(v) = \max_{w \in \text{succ}(v)} \left(\text{height}(w) + \lambda_v(e^{\beta}_{v,w})\right) \tag{3}$$

where $\text{succ}(v) = \{w \mid e^{\beta}_{v,w} \in E\}$ and $\text{height}(v) = 0$ for sink vertices.

These attributes enable priority-based scheduling: depth-descending order implements bottom-up scheduling, while height-descending order implements top-down scheduling.

5.1.2 Bidirectional Timetable. The Bidirectional Timetable (BT) extends Rau's Modulo Resource Reservation Table [31] with bidirectional cell expansion and fine-grained write port conflict detection supporting multi-cycle write-back sequences. We define the BT as T^K_H where H denotes the number of cells per column (i.e., the number of rows), and K denotes the maximum number of machine instructions that can be executed in one cycle. Each cell is indexed by a single integer timeslot $\tau \in \mathbb{Z}$. For an operation u placed in the timetable, we denote its timeslot as $T_H(u)$ or $\tau(u)$ when clear from context. The relationship between timeslot τ and its table position is:

$$\text{row}(\tau) = \tau \bmod H, \quad \text{column}(\tau) = \lfloor \tau / H \rfloor \tag{4}$$

For cyclic scheduling, timeslots wrap around with period H: operations at timeslots τ and $\tau + H$ occupy the same row but different columns, representing different iterations of the modulo schedule. The bidirectional expansion allows τ to grow in both directions (positive or negative integers), accommodating forward and backward scheduling.

To support multi-cycle write-back sequences, BT maintains auxiliary tracking of write port usage. We maintain a set WS of occupied write port moments, where each moment is encoded as a linear index combining functional unit identifier and timeslot modulo H:

$$\text{moment}(f, \tau) = K \times (\tau \bmod H) + f \tag{5}$$

This encoding enables $O(1)$ conflict detection through set membership testing. When an operation u comprising instructions $\{\gamma_0, \ldots, \gamma_{n-1}\}$ is successfully placed at timeslot τ with functional unit assignments $\{f(\gamma_0), \ldots, f(\gamma_{n-1})\}$, the write port occupation set is:

$$\text{WP}(u) = \bigcup_{i=0}^{n-1} \{K \times ((T_H(u) + \rho) \bmod H) + f(\gamma_i) \mid \rho \in \omega^1_{u,v,\alpha}, e^*_{u,v,\alpha} \in \{\text{RAW}, \text{WAW}\}\} \tag{6}$$

where $\omega^1_{u,v,\alpha}$ denotes the write-back moments of operation u on variable α as captured in the SDG edge attributes. Upon successful placement, we update $WS \leftarrow WS \cup \text{WP}(u)$ to record the newly occupied write port moments.

5.1.3 The Three Scheduling Primitives. The SDG and BT structures provide the foundation for three scheduling primitives that encapsulate recurring algorithmic patterns. These primitives serve distinct roles: Primitive 1 defines a design pattern that strategies instantiate with custom priority functions to determine scheduling ordering; Primitives 2 and 3 are reusable functions directly invocable by strategies for hardware constraint validation and timeslot computation, respectively, with their invocation order determined by strategy-specific requirements.

Primitive 1: Determine Scheduling Order. Given unscheduled operations in SDG G, generate a topological ordering respecting precedence constraints while optimizing for a strategy-specific objective. The primitive accepts a priority function $\pi : V \to \mathbb{R}$ and a dependency-filtering predicate $\phi : E \to \{\text{true}, \text{false}\}$. The algorithm iteratively extracts the highest-priority candidate from the set $C = \{v \in V \mid \forall e_{u,v} \in E : \phi(e) \implies u \text{ is scheduled}\}$ until $C = \emptyset$. Common instantiations include height-descending ordering ($\pi(v) = \text{height}(v)$) for top-down scheduling and depth-descending ordering ($\pi(v) = \text{depth}(v)$) for bottom-up scheduling.

Primitive 2: Check Hardware Constraints. Given an operation u and a target timeslot τ in timetable T^K_H, determine whether placing u at τ satisfies architectural constraints and identify a valid functional unit assignment. The notation $u \to T^K_H(\tau)$ denotes an attempt to place operation u in timeslot τ. A successful placement requires satisfying three checks: (1) the total number of

machine instructions does not exceed the cycle capacity K, (2) no parallel execution conflicts or write port conflicts occur, and (3) at least one valid functional unit assignment exists.

We formalize the functional unit assignment problem as a constraint satisfaction problem. Given an operation u with n single instructions, denoted $\gamma_i \in u$ for $i = 0, 1, \ldots, n-1$, let $R := \{r_0, r_1, \ldots, r_{K-1}\}$ represent the set of all functional units available in a cycle, where each unit is identified by a unique integer. For a target timeslot with index τ, the set of available functional units $R_{\text{avail}} \subseteq R$ is the intersection of functional units available across all timeslots in the same row of T_H^K (i.e., all timeslots τ' where $\tau' \bmod H = \tau \bmod H$). The set $\text{res}(\gamma_i) \subseteq R$ denotes the subset of functional units usable by instruction γ_i (specified by instruction capability in the machine model).

The goal is to find a mapping $f : u \to R_{\text{avail}}$ that assigns each γ_i to a functional unit $f(\gamma_i)$, satisfying

- Capability matching: $f(\gamma_i) \in \text{res}(\gamma_i)$ for all $\gamma_i \in u$
- Exclusivity: $f(\gamma_i) \neq f(\gamma_j)$ for any distinct instructions γ_i, γ_j where $i \neq j$
- Write port availability: For each instruction $\gamma_i \in u$, compute its write port occupation set $\text{WP}(\gamma_i)$ using the formula defined in BT. The constraint requires $\text{WP}(\gamma_i) \cap WS = \emptyset$ for all $\gamma_i \in u$.

Upon successful assignment of functional units to all instructions in u, we compute the complete write port occupation set $\text{WP}(u) = \bigcup_{\gamma_i \in u} \text{WP}(\gamma_i)$ and update $WS \leftarrow WS \cup \text{WP}(u)$ as described in the BT structure. We solve this constraint satisfaction problem via backtracking search with early pruning: when assigning γ_i, immediately eliminate candidates that would violate write port constraints for previously assigned instructions, reducing search space before exploring deeper branches.

Primitive 3: Select Timeslot with Relaxation. Given an unscheduled operation v where its predecessors and successors may have been scheduled in timetable T_H, we compute valid placement timeslots through constraint interval intersection. For timing analysis, we define when operations interact with trigger variables. For a scheduled predecessor u and edge $e^{\beta}_{u,v}$, let $\text{def}^{\beta}_u = \tau(u) + \lambda_u(e) - \rho(e) \cdot H$ denote when u completes writing trigger variable β. Similarly, for operation v at candidate timeslot $\tau(v)$, let $\text{use}^{\beta}_v = \tau(v) + \lambda_v(e)$ denote when v reads β.

Each scheduled predecessor u with edge $e^{\beta}_{u,v}$ imposes a Predecessor Constraint Interval (PCI):

$$\text{PCI}(e^{\beta}_{u,v}) = \begin{cases} [\tau(u) + \lambda_u(e) - \lambda_v(e) - \rho(e) \cdot H, \\ \quad \tau(u) + \lambda_u(e) - \lambda_v(e) - \rho(e) \cdot H + H) & \text{if } \delta(e) = \text{RAW} \\ (\tau(u) + \lambda_u(e) - \lambda_v(e) - \rho(e) \cdot H, +\infty) & \text{otherwise} \end{cases} \tag{7}$$

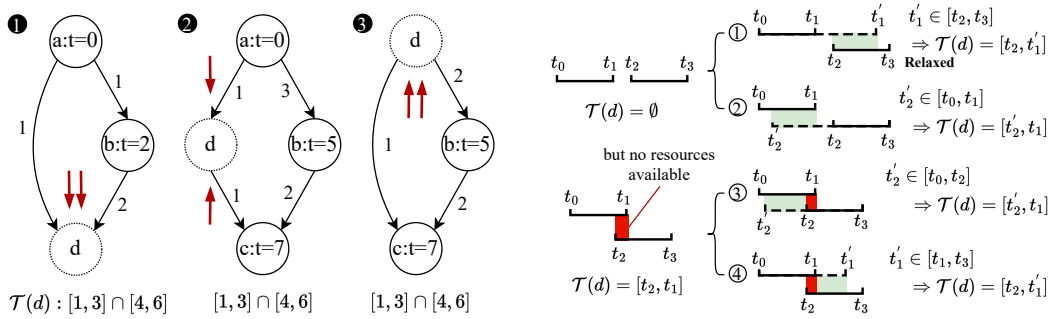
Each scheduled successor w with edge $e^{\beta}_{v,w}$ imposes a Successor Constraint Interval (SCI):

$$\text{SCI}(e^{\beta}_{v,w}) = \begin{cases} (\tau(w) - \lambda_v(e) + \lambda_w(e) + \rho(e) \cdot H - H, \\ \quad \tau(w) - \lambda_v(e) + \lambda_w(e) + \rho(e) \cdot H] & \text{if } \delta(e) = \text{RAW} \\ (-\infty, \tau(w) - \lambda_v(e) + \lambda_w(e) + \rho(e) \cdot H) & \text{otherwise} \end{cases} \tag{8}$$

Valid timeslots for operation v emerge from intersecting all constraints:

$$\mathcal{T}(v) = \left(\bigcap_{e^{\beta}_{u,v} \in \text{pred-edges}(v)} \text{PCI}(e^{\beta}_{u,v}) \right) \cap \left(\bigcap_{e^{\beta}_{v,w} \in \text{succ-edges}(v)} \text{SCI}(e^{\beta}_{v,w}) \right) \tag{9}$$

Dependency relaxation. Aggressive scheduling frequently encounters infeasibility: either conflicting dependencies yield $\mathcal{T}(v) = \emptyset$ (Figure 5a), or all valid timeslots fail resource allocation (Primitive 2). Traditional modulo schedulers [22, 23] respond by incrementing the initiation interval



(a) Three cases of $\mathcal{T}(d) = \emptyset$ ($H = 3$).

(b) Relaxation strategies for extending variable lifetimes.

Fig. 5. Dependency relaxation mechanism. **(a)** Dotted circles: unscheduled operations; solid circles: scheduled operations; red arrows: dependencies on trigger variables with latency. ❶ Dual-predecessor conflict on a trigger variable. ❷ Predecessor-successor conflict. ❸ Dual-successor conflict. t = [value] indicates the emission timing of each operation. **(b)** Let $[t_0, t_1]$ denote a PCI and $[t_2, t_3]$ denote an SCI on the trigger variable β. Upper row: Scenario I addresses conflicting constraints from (a) by extending β's lifetime—(①)relax PCI backward, (②)relax SCI forward. Lower row: Scenario II addresses resource depletion by increasing scheduling opportunities—(③)relax SCI forward, (④)relax PCI backward.

and restarting, often degrading throughput significantly. We propose an alternative: selectively extend trigger variable lifetimes through move operations, trading modest instruction overhead for maintained schedule compactness—a capability absent from existing schedulers. The mechanism operates through a three-step workflow:

Step 1: Compute Lifetime Intervals. For each variable β on a RAW edge $e^{\beta}_{u,v}$, compute the production time $t_p(\beta) = \text{def}^{\beta}_u = \tau(u) + \lambda_u(e) - \rho(e) \cdot H$ (when the defining instruction u completes writing β) and consumption time $t_c(\beta) = \text{use}^{\beta}_v = \tau(v) + \lambda_v(e)$ (when consumer v reads β). The feasibility condition requires: $t_p(\beta) \le t_c(\beta)$, i.e., the producer must complete before the consumer reads.

Step 2: Identify Infeasibility. When placement decisions force $\mathcal{T}(v) = \emptyset$ or all valid timeslots fail resource checks, standard scheduling fails. Rather than abandoning the schedule, identify the timing gap: $\Delta = t_p(\beta) - t_c(\beta)$ when $\Delta > 0$. Figure 5a illustrates three infeasibility cases: dual-predecessor conflict (❶), predecessor-successor conflict (❷), and dual-successor conflict (❸).

Step 3: Construct Move Chain. Insert a chain of move instructions that extends the variable's lifetime to bridge the timing gap. Figure 5b illustrates our relaxation strategies, operating exclusively on flow dependencies (RAW edges). For predecessor-dominated conflicts (Cases ①④), PCI relaxes backward: $[t_0, t_1 + \Delta]$. For successor-dominated conflicts (Cases ②③), SCI relaxes forward: $[t_2 - \Delta, t_3]$. This directional design ensures correctness: PCIs extend toward later timeslots (propagating trigger variables longer from producers), while SCIs extend toward earlier timeslots (allowing more time before consumers read).

When the lifetime span exceeds one modulo period ($D = \lfloor (\text{use}^{\beta}_v - \text{def}^{\beta}_u)/H \rfloor > 0$), we insert D move operations $\{m_1, \ldots, m_D\}$ forming propagation chain $u \xrightarrow{\beta} m_1 \xrightarrow{t_1} \cdots \xrightarrow{t_{D-1}} m_D \xrightarrow{t_D} v$ through temporary variables $\{t_0 = \beta, t_1, \ldots, t_D\}$. The chain length is $\lceil \Delta/\lambda_{\text{mov}} \rceil$ where λ_{mov} denotes the move operation latency. Each move m_i must be placed within its valid timeslot interval:

$$\mathcal{T}(m_i) = \begin{cases} [E(\text{use}^{t_i}_{m_i,m_{i+1}}), L(\text{use}^{t_i}_{m_i,m_{i+1}}, i)] & \text{if } 1 \le i < D \\ [E(\text{use}^{\beta}_v), L(\text{use}^{\beta}_v, D)] & \text{if } i = D \end{cases} \tag{10}$$

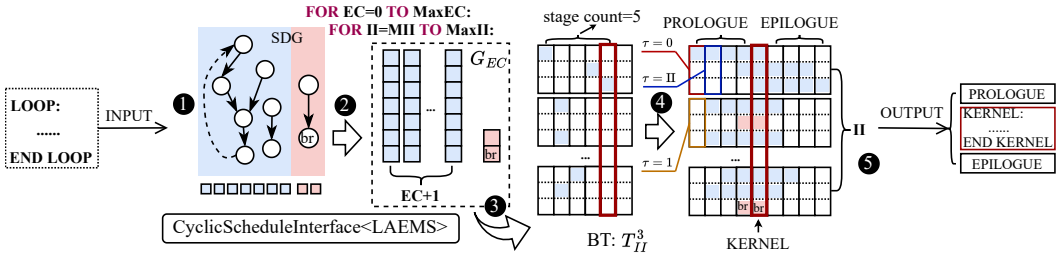


Fig. 6. LAEMS pipelining workflow. ❶ SDG construction → ❷ (EC, II) exploration with SDG expansion (excluding loop-counting chain) → ❸ Alternating scheduling (see Algorithm 3) → ❹ Timetable completion (unfolding operations and inserting loop-counting chains) → ❺ Output the instruction bundles in order from the non-empty cell with smallest index value (τ) and make KERNEL as a loop structure.

where $E(x) = x - H - \lambda_{\text{mov}} + 1$ and $L(x,k) = \min(\text{def}_u^\beta + k \times H - 1, x - \lambda_{\text{mov}})$. This partitions the original interval $[\text{def}_u^\beta, \text{use}_v^\beta]$ into subintervals, each satisfying local producer-consumer timing.

Correctness guarantees. Program semantics preservation relies on three invariants:

Invariant 1 (Applicability restriction): Relaxation targets only RAW edges with explicit producer-consumer relationships. Anti-dependencies (WAR), output dependencies (WAW), and control dependencies (AJ, BJ) remain inviolable—they encode resource conflicts and sequencing constraints fundamental to correct execution.

Invariant 2 (Data propagation completeness): The move chain construction ensures each move m_i reads t_{i-1} only after its producer completes, and completes before the successor reads. The happens-before relationship is maintained through compositional interval partitioning.

Invariant 3 (Resource constraint satisfaction): Framework commits relaxation only when Primitive 2 successfully validates resource constraints for all inserted moves. If any move fails placement due to insufficient functional units or register pressure violations, the entire relaxation is rejected, forcing alternative strategies or interval incrementation. Since the SDG (Section 5.1.1) captures all instruction dependencies, and relaxation applies only to RAW edges with validated move chains, program semantics are never compromised by unimplementable schedules.

5.2 Longevity-Aware Expanded Modulo Scheduling

Expanded Modulo Scheduling (EMS) [23] relaxes the integer constraint by jointly optimizing Expansion Count (EC) and Initiation Interval (II), enabling non-integer effective IIs through loop unrolling. EMS schedules operations from different unrolled layers in an interleaved manner to maximize resource utilization. However, EMS addresses only simple innermost loops with regular memory access patterns and lacks mechanisms to prevent scheduling failures when aggressive pipelining creates tight constraint combinations. LAEMS extends EMS with framework-level support for realistic loop structures (Section 5.2.1) and algorithmic improvements that reduce scheduling failures (Section 5.2.2).

Figure 6 illustrates the LAEMS core pipeline. Given a pipelinable loop, LAEMS first constructs an SDG capturing loop-body dependencies, then explores the (EC, II) parameter space by expanding the SDG for each candidate pair. For each configuration, an alternating scheduling strategy places operations into the timetable, after which unfolding replicates instructions across kernel columns. The pipeline emits instruction bundles in timeslot order, encapsulating the steady-state kernel within a loop structure. Section 5.2.3 gives an illustrative example of this process.

Algorithm 2: Proactive Store Reordering

Input: Operation list L, timetable T_{II}, expanded SDG G_{EC}
Output: Reordered L if beneficial
Notation:
I_i^ℓ: The i-th operation in the ℓ-th loop expansion layer
$L = \{I_0^0, I_1^0, \ldots, I_{n-1}^0; I_0^1, \ldots, I_{n-1}^1; \ldots; I_0^{\text{EC}}, \ldots, I_{n-1}^{\text{EC}}\}$: Structured list where ";" separates expansion layers
n: Number of operations per layer (one iteration before expansion)
EC: Expansion count (number of unrolled copies minus one)
U_i: Store operation I_i across all layers $\{I_i^0, I_i^1, \ldots, I_i^{\text{EC}}\}$
D_i: EFT differences $\{\text{EFT}(I_i^\ell) - \text{EFT}(I_i^{\ell-1}) \mid \ell \in [1, \text{EC}]\}$
$\text{EFT}(S)$: Earliest Feasible Timeslot for store S, computed as $\min \mathcal{T}(S)$
$\text{IsStore}(I)$: Predicate true if instruction I is a store operation
$\text{sign}(x)$: Sign function: $+1$ if $x > 0$, -1 if $x < 0$, 0 otherwise
$S \to T_{\text{II}}(\cdot)$: Operation S can be placed in T_{II} at some valid timeslot

for $i = 0, 1, \ldots, n-1$ **do**
  if $\neg\text{IsStore}(I_i^0)$ **then continue**
  $U_i \leftarrow \{I_i^0, I_i^1, \ldots, I_i^{\text{EC}}\}$ `// Store ops across layers`
  $D_i \leftarrow \{\text{EFT}(I_i^\ell) - \text{EFT}(I_i^{\ell-1}) \mid \ell \in [1, \text{EC}]\}$
  if $(\forall \ell : I_i^\ell \to T_{\text{II}}(\cdot)$ feasible$) \wedge \sum \text{sign}(D_i) = 1$ **then**
    leap $\leftarrow \arg_\ell(D_{i,\ell} > 0)$, for $\ell = 1, 2, \ldots, \text{EC}$
    $L_a \leftarrow \{I_j^\ell \mid j \in [0, n), \ell \in [0, \text{leap}]\}$
    $L_b \leftarrow \{I_j^\ell \mid j \in [0, n), \ell \in [\text{leap}+1, \text{EC}]\}$
    Update L: reorder as $L_b; L_a$
return L

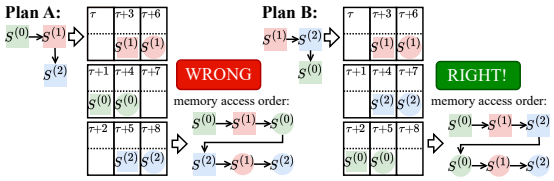


Fig. 7. Store operation reordering prevents scheduling failures. **Plan A**: Original height-based scheduling places $S^{(0)}$, $S^{(1)}$, and $S^{(2)}$ in sequence, leading to ordering violations. **Plan B**: Proactive reordering yields a valid schedule by exploiting resource availability patterns. Initial operations are marked by squares, and their copies are marked with circles of the same color.

Algorithm 3: Alternating Scheduling

Input: Timetable T_{II}, expanded SDG G_{EC}
Output: Updated T_{II} with all operations scheduled
Notation:
Q: Queue of unscheduled operations
L: List of operations with largest height (current scheduling layer)
P: Set of postponed non-critical operations
$\text{height}(v)$: Maximum latency-weighted distance from operation v to sink (computed in SDG as defined in Section 5.1.1)
Critical operation: Operation on the critical path with zero slack:
$\text{depth}(v) + \text{height}(v) = \text{depth}_{\max}$
Critical successor of X: Successor operation of X that is critical
$\text{succ}(v)$: Set of successor operations of v in G_{EC}
$\text{Extract}(X, S)$: Remove operation X from set S and return X
$\text{PLACE}(X)$: Schedule operation X using Primitives 2-3 (Section 5.1):
1. Compute valid timeslots $\mathcal{T}(X)$ via Primitive 3
2. Apply relaxation if $\mathcal{T}(X) = \emptyset$ or all slots fail resources
3. Validate resource constraints via Primitive 2: $X \to T_{\text{II}}(\tau)$
4. Select $\tau \in \mathcal{T}(X)$ closest to scheduled boundary
5. Terminate if placement fails after relaxation

$\text{ReorderStoreOps}(L, T, G)$: Apply Algorithm 2 to detect and correct store operation ordering conflicts in list L

$Q \leftarrow$ all unscheduled operations in G_{EC}
$P \leftarrow \emptyset, \quad L \leftarrow \emptyset$
while $Q \neq \emptyset$ **do**
  $h_{\max} \leftarrow \max\{\text{height}(v) \mid v \in Q\}$
  $L \leftarrow \{v \in Q \mid \text{height}(v) = h_{\max}\}$
  $Q \leftarrow Q \setminus L$
  $\text{ReorderStoreOps}(L, T_{\text{II}}, G_{\text{EC}})$ `// Algorithm 2`
  `// Phase 1: Schedule critical operations first`
  while $\exists v \in L : v$ is critical **do**
    $X \leftarrow$ Extract operation with fewest successors from L
    $\text{PLACE}(X)$
    `// Check if postponed ops can now be scheduled`
    for $Y \in P$ **do**
      $S_{\text{crit}}(Y) \leftarrow \{w \in \text{succ}(Y) \mid w \text{ is critical}\}$
      if $S_{\text{crit}}(Y) \subseteq \{\text{scheduled operations}\}$ **then**
        $Y \leftarrow \text{Extract}(Y, P)$
        $\text{PLACE}(Y)$
  `// Phase 2: Schedule or postpone non-critical ops`
  while $L \neq \emptyset$ **do**
    $Z \leftarrow$ Extract any non-critical operation from L
    $S_{\text{crit}}(Z) \leftarrow \{w \in \text{succ}(Z) \mid w \text{ is critical}\}$
    if $S_{\text{crit}}(Z) \subseteq \{\text{scheduled operations}\}$ **then**
      $\text{PLACE}(Z)$
    else
      $P \leftarrow P \cup \{Z\}$

5.2.1 Framework-Level Extensions. LASM enables LAEMS to handle complete programs with diverse loop patterns. Production loops exhibit characteristics that EMS does not handle: cumulative variables arising from reduction operations (e.g., $\sum_{i=0}^{N-1} a[i]$), updated-only variables from selection operations (e.g., $\max_{i=0}^{N-1} a[i]$), and variable iteration counts requiring runtime remainder handling.

After expansion to EC copies, each layer maintains separate partial results. For cumulative variables, LAEMS inserts EC merging operations in the epilogue: $s \leftarrow s^{(0)} + s^{(1)} + \cdots + s^{(\text{EC})}$. For updated-only variables, the epilogue performs EC selection operations: $m \leftarrow \max(m^{(0)}, \ldots, m^{(\text{EC})})$. These operations are scheduled using Primitive 1 with bottom-up ordering. For variable iteration counts, LAEMS generates two code paths: a pipelined kernel for the main iterations and a remainder loop (scheduled separately) handling N mod EC iterations.

5.2.2 Algorithm-Level Extensions. LAEMS introduces three algorithmic innovations that address fundamental limitations in the original EMS design while enhancing performance and reducing time overhead.

Proactive Store Reordering. Expanded modulo scheduling must preserve memory access ordering across unrolled layers. For store operation S expanded to $S^{(0)}, \ldots, S^{(\mathrm{EC})}$, EMS enforces strict temporal ordering:

$$\forall k \in [1, \mathrm{EC}] : \max_{j<k} \tau(S^{(j)}) \leq \tau(S^{(k)}) \leq \min_{j<k} \tau(S^{(j)}) + \mathrm{II} - 1 \tag{11}$$

When this constraint is violated, EMS discards the entire schedule and restarts with an increased II. LAEMS observes that height-based priority ordering can produce suboptimal sequences for store operations, as dependency-based priorities often mismatch resource-availability patterns.

Algorithm 2 presents the detection and correction process. The algorithm uses the notation I_i^ℓ to denote the i-th operation in the ℓ-th expansion layer. Before scheduling store operations, LAEMS computes the earliest feasible timeslot $\mathrm{EFT}(I_i^\ell) = \min \mathcal{T}(I_i^\ell)$ for each layer based purely on data dependencies. If the sequence exhibits exactly one *leap point* where $\mathrm{EFT}(I_i^\ell) < \mathrm{EFT}(I_i^{\ell-1})$ (detected by $\sum \mathrm{sign}(D_i) = 1$ in line 5), and sufficient functional units exist to schedule all stores, LAEMS preemptively reorders the scheduling sequence: layers $[\mathrm{leap}+1, \mathrm{EC}]$ are scheduled before layers $[0, \mathrm{leap}]$. As illustrated in Figure 7, this reordering prevents failures without compromising the initiation interval. The key insight is scheduling-order independence: while the final schedule must satisfy Equation (11), the order in which we attempt to place operations can be optimized based on resource availability patterns rather than purely dependency-based priorities.

Longevity-Aware Dependency Relaxation. Aggressive pipelining with small II often leaves valid timeslots empty for some operation v due to tight dependencies and limited resources. Traditional modulo schedulers respond by incrementing the II and restarting. LAEMS instead employs the relaxation technique provided by Primitive 3 (Section 5.1.3) that extends variable lifetimes to enable placement at timeslots that would otherwise violate data dependencies.

When the scheduler attempts to place operation v and encounters $\mathcal{T}(v) = \emptyset$ or all timeslots fail resource checks, it invokes Primitive 3's relaxation operation. The critical insight is that relaxation trades move operations for throughput: rather than incrementing the II (reducing the frequency of instruction emission of the pipelined kernel), LAEMS inserts several move operations consuming functional units but maintaining the tight pipeline. LAEMS applies relaxation selectively through cost-benefit analysis, verifying that the target timeslot has available functional units and the estimated register pressure increase remains acceptable.

Algorithm 3 presents the core scheduling procedure orchestrating the above two innovations. The algorithm extends traditional height-based list scheduling with critical path awareness and strategic operation postponement.

Before scheduling begins, LAEMS identifies the critical path in G_{EC}: the longest latency-weighted path from any source to any sink vertex. An operation v is *critical* if it lies on this path, equivalently characterized by zero slack: $\mathrm{depth}(v) + \mathrm{height}(v) = \mathrm{depth}_{\max}$, where $\mathrm{depth}_{\max} = \max_{u \in G_{\mathrm{EC}}} \mathrm{depth}(u)$ represents the length of the critical path. Critical operations directly determine the schedule length and receive scheduling priority in Algorithm 3.

The algorithm maintains three data structures: Q (unscheduled operations), L (current operations to be scheduled), and P (postponed non-critical operations). In each iteration, operations with the largest height value transfer from Q to L, ensuring fair treatment across expansion layers. Before scheduling, Algorithm 2 proactively reorders store operations within L to prevent ordering violations. The `PLACE` operation, implemented using Primitives 2 and 3, attempts to schedule each operation at its best timeslot. Critical operations (those on the critical path) schedule immediately, while non-critical operations may postpone to P until their critical successors complete.

Parallel Search Across Expansion Counts. The nested iteration structure of LAEMS (outer loop over EC, inner loop over II) exhibits an exploitable property. For distinct expansion counts

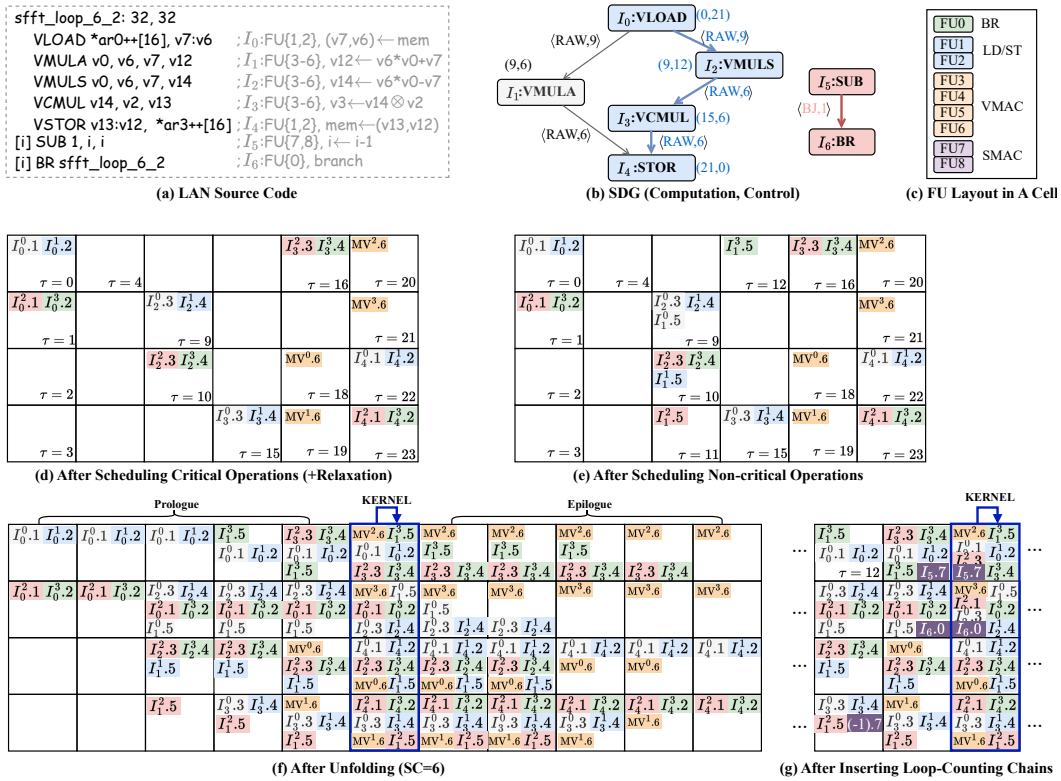


Fig. 8. LAEMS illustrative example: a loop from FFT kernel with EC=3 and II=4. EC=3 expands the loop into four layers ($\ell = 0, \ldots, 3$); II=4 starts successive pipelined super-iterations four cycles apart. (a) LAN source code with operation indices (I_i), available FUs, and semantics. (b) SDG with edge labels $\langle type, \lambda \rangle$ and node annotations (depth, height); blue path indicates critical operations. (c) Cell structure showing 9 functional units (FU0–FU8) available per cycle. (d,e) Timetables after alternating scheduling with dependency relaxation (4 VMOV inserted). Panels (d)–(g) use the same timetable notation: header entries label schedule regions and issue-time positions, empty cells denote idle timetable slots, and each non-empty cell value $I_i^{\ell}.k$ denotes operation I_i from expansion layer ℓ assigned to FUk. (f) Timetable after unfolding with SC=6 stages; the Prologue, KERNEL, and Epilogue headers mark the three unfolded schedule regions, and the dashed box marks the steady-state kernel. (g) Final timetable after inserting the loop-counting chain, where the inserted I_5 and I_6 entries denote SUB and BR.

EC_i and EC_j where $i \neq j$, the search for minimum feasible II proceeds independently since each (EC, II) pair produces a self-contained schedule with its own resource allocation. LAEMS exploits this independence through parallel search across expansion counts. The framework distributes EC values among worker threads, with each worker sequentially iterating over IIs for its assigned EC. Once a worker successfully generates a schedule, it submits the candidate to a thread-safe queue.

5.2.3 Illustrative Example. We illustrate the LAEMS core workflow (Figure 6) using a 32-iteration loop from an FFT row processing kernel. The loop body contains five computational operations: VLOAD (vector load, λ=9), VMULA (vector multiply-add, λ=6), VMULS (vector multiply-subtract, λ=6), VCMUL (complex multiply, λ=6), and VSTOR (vector store, λ=1). The loop-counting chain comprises SUB (λ=1) and BR (conditional branch, λ=7). VMOV (vector move, λ=1) may be inserted during

scheduling to extend variable lifetimes when dependency relaxation is invoked. In this example, each beat provides 9 functional units $\{FU_0, FU_1, \ldots, FU_8\}$. Figure 8(c) shows the per-beat FU layout: VLOAD and VSTOR can use FU1 or FU2; VMULA, VMULS, VCMUL, and VMOV can use FU3–FU6; SUB can use FU7 or FU8; BR uses FU0.

❶ **SDG Construction.** LAEMS constructs the SDG $G = (V, E)$ from the loop body, computing depth(v) and height(v) for each vertex using Equations (2, 3). Figure 8(b) shows the result: five computational vertices (I_0 through I_4) plus the loop-counting chain (I_5: SUB, I_6: BR). Each edge is labeled with its dependency type and latency in the form $\langle type,\lambda \rangle$. The critical path is $I_0 \rightarrow I_2 \rightarrow I_3 \rightarrow I_4$ (i.e., VLOAD → VMULS → VCMUL → VSTOR) with $d_{\max} = 21$. An operation v is *critical* if depth(v) + height(v) = $d_{\max}$; thus I_1 (VMULA) with (9, 6) is non-critical since $9 + 6 = 15 \neq 21$.

❷ **SDG Expansion and Parallel Search.** LAEMS explores multiple expansion counts in parallel. For EC=3, the 32-iteration loop executes as $32/(3+1) = 8$ pipelined super-iterations across 4 layers ($\ell \in \{0, 1, 2, 3\}$). The expanded operation set is: $L = \{I_0^0, I_1^0, \ldots, I_4^0; I_0^1, I_1^1, \ldots, I_4^1; \ldots; I_0^3, I_1^3, \ldots, I_4^3\}$ where I_i^ℓ denotes operation i in layer ℓ, and semicolons separate layers. This expansion annotates each operation with a layer index without physically replicating the graph structure. The bidirectional timetable $T^9_{\text{II}=4}$ is initialized with $H = 4$ rows, where each timeslot τ satisfies row(τ) = τ mod 4. Notably, the loop-counting chain is excluded from expansion. The EC/II pair in Figure 8 is the output of LAEMS search, not a manually fixed parameter choice. LAEMS enumerates EC candidates and, for each EC, searches II upward from the minimum feasible bound. For this loop, EC=3 creates four expansion layers and II=4 creates a four-row modulo timetable. Without relaxation, the candidate EC=3, II=4 is infeasible because some VSTOR instances have no available timeslots. Dependency relaxation makes these stores placeable, so LAEMS keeps II=4 instead of increasing II to 5.

❸ **Alternating Scheduling.** Algorithm 3 schedules operations (excluding the loop-counting chain) in height-descending order, processing operations across all layers at each height level. Before each height group, Algorithm 2 checks store operation ordering. With only one store per iteration here, no reordering is triggered.

Operations are placed in height order: first the critical operations ($I_0 \rightarrow I_2 \rightarrow I_3 \rightarrow I_4$), then non-critical I_1. For each I_i^ℓ, Primitive 3 computes valid timeslots $\mathcal{T}(I_i^\ell)$ by intersecting dependency constraints with resource availability. The VLOAD instances $\{I_0^0, I_0^1, I_0^2, I_0^3\}$ are placed at $\tau \in \{0, 1, 2, 3\}$; VMULS instances at $\tau \in \{9, 10, 11, 12\}$; VCMUL instances at $\tau \in \{15, 16, 17, 18\}$. LAEMS does not depend on the loop expansion layer during placement. It relies on the dependence graph and the modulo timetable constraints.

When scheduling VSTOR instances, a conflict emerges: with II=4, the feasible set $\mathcal{T}(I_4^\ell) = \emptyset$ for some layers due to tight constraints from VMULA. Standard EMS would increment II to 5. Instead, LAEMS invokes Primitive 3's relaxation mechanism. For the constraining RAW dependency from I_1 (VMULA) to I_4 (VSTOR) through v12, LAEMS applies the following local code transformation:

Before: I_1^ℓ: `VMULA v0, v6, v7, v12` → I_4^ℓ: `VSTOR v13:v12, *ar3++[16]`
After: I_1^ℓ: `VMULA v0, v6, v7, v12` → M^ℓ: `VMOV v12, t` → I_4^ℓ: `VSTOR v13:t, *ar3++[16]`.

The inserted VMOV forwards v12 into compiler-generated temporary t. This extends the value lifetime across the conflicting interval while preserving the RAW order and the store address. The transformed code makes the store placement feasible at II=4. Figure 8(d,e) shows the resulting timetable state.

❹ **Timetable Completion.** After all computational operations are scheduled, LAEMS unfolds the modulo schedule by replicating each operation (SC−1) times at II-spaced intervals, generating the complete prologue-kernel-epilogue structure. The SC refers to the stage count. Here, SC= $\lceil (d_{\max} + 1)/\text{II} \rceil = \lceil 22/4 \rceil = 6$. Figure 8(f) shows the timetable after unfolding, where the kernel region contains the steady-state operations that will form the pipelined loop body.

Then, LAEMS inserts the loop-counting chain. The key timing constraint is that the branch instruction (I_6: BR) must issue early enough for its delayed effect to redirect control at the kernel's end. Given BR's latency $\lambda_{\text{BR}} = 7$ and the kernel ending at timeslot $\tau_{\text{ker,end}}$, the earliest BR slot satisfies:

$$\tau_{\text{BR}} = \tau_{\text{ker,end}} - \lambda_{\text{BR}} + 1 \tag{12}$$

LAEMS places I_6 (BR) at this computed slot, then places I_5 (SUB, which updates the loop counter) one cycle earlier to satisfy the RAW dependency. The chain is then replicated at II-spaced intervals within the kernel region. Figure 8(g) shows the final timetable with the loop-counting chain inserted. Because the kernel's loop semantics are consistent with `do {} while();`, an additional SUB instruction must be added before entering the kernel to decrement the loop counter by 1.

❺ **Output Generation.** Instruction bundles are emitted in ascending timeslot order ($\tau = \ldots \rightarrow 0 \rightarrow 1 \rightarrow \ldots \rightarrow 43 \rightarrow \ldots$). Each bundle packs operations at the same beat according to their functional unit assignments. The kernel becomes a loop structure with BR branching back to its entry point.

As a result, LAEMS maintains II=4 instead of incrementing to II=5. For 32 iterations: the baseline (EC=3, II=5) achieves 83 cycles without relaxation, while LAEMS (EC=3, II=4) achieves 76 cycles with relaxation—an 8.43% improvement by trading 4 VMOV instructions for sustained throughput.

5.2.4 Integration into Strategy Library. LAEMS is integrated into the strategy library via the `CyclicScheduleInterface`. Given user-specified EC bounds, for example $[0, 8]$, the framework explores multiple unrolling factors. For each EC value, scheduling starts from the theoretical minimum II and proceeds by iterating over feasible II values, producing a fixed number of candidate schedules, with 10 used by default. These schedules are subsequently submitted to the competitive selection mechanism described in Section 4.2.

5.3 Front-Back Block Fusion

The progressive scheduling pipeline (Section 4) processes code regions in decreasing scope order, creating optimization opportunities at boundaries between scheduled and unscheduled regions. After cyclic scheduling completes, loops exist as fully scheduled beat blocks, but adjacent basic blocks remain unscheduled. Traditional linear schedulers process only the unscheduled region, lacking visibility into adjacent scheduled code, leaving potential parallelism across optimization boundaries unexploited. Aggressive loop pipelining produces compact kernels where instructions complete at varying times due to heterogeneous latencies, creating delay slots before loop exits. Meanwhile, adjacent basic blocks often contain independent operations that could execute during these slots.

Front-Back Block Fusion (FBBF) addresses this through cross-boundary scheduling that bridges scheduled and unscheduled regions. The key insight is that SAS's mixed scheduling state enables constructing a unified dependence view spanning both states. It allows operations to move across optimization boundaries, thus filling delay slots without violating dependencies or resource constraints. FBBF captures such opportunities by analyzing cross-boundary dependencies to determine fusion direction, then applying unified scheduling primitives to perform placement.

Importantly, FBBF does not fuse basic blocks in the traditional compiler sense (i.e., merging two blocks into one by eliminating intervening jumps). Instead, the "front" region refers to code that has already completed scheduling (typically loop kernels as beat blocks), while the "back" region refers to adjacent code awaiting scheduling (basic blocks). FBBF examines whether instructions from back-region basic blocks can be interleaved into unused execution slots within the front-region's schedule, subject to dependency and resource constraints. This cross-boundary interleaving exploits parallelism that traditional single-region schedulers cannot capture.

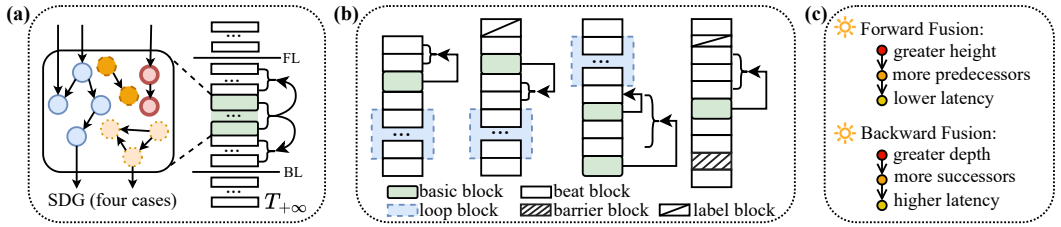


Fig. 9. Front-back block fusion workflow. **(a)** Conceptual SDG showing four connected components representing all possible dependency patterns between basic blocks (green) and beat blocks (white). FL/BL mark impenetrable boundaries. **(b)** Fusion direction selection for each component based on cross-boundary dependency counts. **(c)** Multi-level priority scheme for determining the scheduling order of operations.

5.3.1 Three-Phase Scheduling Workflow. FBBF orchestrates scheduling through three phases.

Phase 1: Identify Fusion Candidates. For a target basic block B_{target}, FBBF traverses the SAS bidirectionally to collect consecutive basic blocks $\{B_{-m}, \ldots, B_{-1}, B_{\text{target}}, B_1, \ldots, B_n\}$ that form a contiguous unscheduled region. Traversal terminates at impenetrable boundaries: loop block entries/exits (which encapsulate modulo-scheduled kernels), label blocks marking branch targets (which serve as synchronization points for control flow), and barrier blocks requiring optimization isolation. Let FL (forward limit) and BL (backward limit) denote the nearest boundaries.

FBBF then collects scheduled beat blocks adjacent to this region. For the forward direction, it gathers up to $d_{\max}$ consecutive beat blocks immediately preceding FL; for the backward direction, it gathers up to $d_{\max}$ beat blocks immediately following BL. The depth threshold $d_{\max}$ is set to the maximum instruction latency in the target ISA (typically 9–12 cycles). This ensures that the full contextual dependencies are fetched. As illustrated in Figure 9(a), the active fusion region contains basic blocks and beat blocks, with potential dependencies spanning both regions.

Phase 2: Dependency Analysis and Direction Selection. FBBF constructs an SDG spanning all operations in the active region. The left part of Figure 9(a) shows a conceptual SDG with four connected components, showing all possible associations with the context. For each connected component C, FBBF counts cross-boundary edges:

$$\begin{aligned} \text{ForwardDep}(C) &= |\{e_{u,v} \in C \mid u \text{ scheduled}, v \text{ unscheduled}\}| \\ \text{BackwardDep}(C) &= |\{e_{u,v} \in C \mid u \text{ unscheduled}, v \text{ scheduled}\}| \end{aligned} \tag{13}$$

As shown in Figure 9(b), when $\text{ForwardDep}(C) > \text{BackwardDep}(C)$, forward fusion schedules operations into delay slots preceding FL; otherwise, backward fusion schedules into slots following BL. This greedy heuristic maximizes already-established timing constraints, reducing placement failures during scheduling.

Phase 3: Fusion via Unified Primitives. Different connected components schedule independently, enabling parallel exploration of multiple fusion opportunities. For each component C, FBBF initializes a timetable $T_{+\infty}$ by copying relevant scheduled beats, then schedules unscheduled operations from C using the three primitives. Primitive 1 establishes operation ordering via a multi-level priority scheme (Figure 9(c)). For each operation v in priority order, Primitive 3 computes valid timeslots $\mathcal{T}(v)$ and applies relaxation when necessary to expand the candidate set. Primitive 2 validates resource constraints and selects timeslots closest to the scheduled boundary to maximize delay slot filling. FBBF applies relaxation conservatively, favoring schedules that maximize delay slot filling with minimal moving overhead.

Table 1. Benchmark suite characteristics spanning diverse computational patterns across embedded and HPC domains.

Program	Domain	Description	Input Shape	Pattern
gemm	HPC	Matrix-matrix product $C \leftarrow C - AB$	$A(\mathrm{M},\mathrm{K}), B(\mathrm{K},\mathrm{N}), C(\mathrm{M},\mathrm{N})$	Dense, regular
gemvt	HPC	Matrix-vector product $Y \leftarrow \alpha A^\top X + \beta Y$	$A(\mathrm{M},\mathrm{N}), X(\mathrm{N},1), Y(\mathrm{M},1)$	Dense, regular
asum	Embedded	Absolute sum $\sum_{i=0}^{\mathrm{N}-1} \lvert X[i]\rvert$	$X(\mathrm{N},1)$	Vector, reduction
axpby	Embedded	Linear combination $Y \leftarrow \alpha X + \beta Y$	$X(\mathrm{N},1), Y(\mathrm{N},1)$	Vector, arithmetic
scal	Embedded	Scalar multiply $X \leftarrow \alpha X$	$X(\mathrm{N},1)$	Vector, arithmetic
dot	Embedded	Dot product $Y \leftarrow X \cdot Y$	$X(\mathrm{N},1), Y(\mathrm{N},1)$	Vector, reduction
fft_col	Embedded	Column-based DFT	$X(\mathrm{N},1), A(\mathrm{N},32)$	Transform, complex
fft_r2	Embedded	Row-based DFT radix-2	$X(\mathrm{N},1), W(1,\mathrm{N}+32)$	Transform, complex
SpMV	HPC	Sparse matrix-vector multiply	$A(\mathrm{nir},\mathrm{lr}), X(\mathrm{lr},1)$	Sparse, irregular
coeff_update	Embedded	Coefficient update $A[i,j] \leftarrow A[i,j] \times X[i]$	$X(\mathrm{N},1), A(\mathrm{N},32)$	Filter, dependent
mul_coeff	Embedded	Element-wise product $A[i,j] \leftarrow A[i,j] \times B[i,j]$	$A(\mathrm{N},32), B(\mathrm{N},32)$	Filter, arithmetic
prolongation	HPC	Multigrid prolongation	$X(\mathrm{lr},1), Y(\mathrm{lr},1)$	Stencil, structured
idamax	Embedded	Maximum absolute index	$X(\mathrm{N},1)$	Search, dependent

5.3.2 Integration into Strategy Library. After attempting all operations, FBBF submits the resulting schedule to compete with alternatives generated by other strategies. FBBF integrates through both `CyclicScheduleInterface` and `LinearScheduleInterface`. Through the cyclic interface, it provides an alternative for simple loops where aggressive pipelining proves ineffective. Through the linear interface, it is used to fully utilize delay slots.

6 Experiments

This experimental evaluation addresses four research questions. RQ1 evaluates scheduling quality and multi-strategy effectiveness against hand-optimized assembly. RQ2 analyzes compilation time overheads and parallel exploration efficiency. RQ3 assesses engineering cost for developers and strategy designers. RQ4 validates real-world applicability on full-scale applications beyond isolated kernels.

6.1 Experimental Setup

Implementation and Environment. The LASM framework comprises 42,138 lines of C++17 and runs on an AMD Ryzen AI 9 HX 370 (12-core, 24-thread, operating at ~4.0 GHz) with 32 GB memory running Ubuntu 22.04.5 LTS.

Target Platforms. We evaluate our framework for two production VLIW processors representing distinct architectural designs. Matrix DSP [8] is a 32-bit VLIW DSP, while MT3000-GPDSP [7] is a 64-bit VLIW accelerator from a heterogeneous processor. Both employ a Vector-SIMD architecture with scalar processing units (SPUs) and vector processing units (VPUs). Each VPU consists of 16 vector processing elements (VPEs) operating in SIMD mode. On Matrix DSP, each VPE provides 64 32-bit registers and 4 fused multiply-add units (FMACs), delivering one FP32 multiply-add operation per cycle, for a total of 128 FLOPs per cycle. On MT3000-GPDSP, each VPE provides 64 64-bit registers and 3 FMACs for FP64 computation, delivering 96 FLOPs per cycle.

Benchmarks. Table 1 presents 13 typical workloads spanning embedded signal processing and HPC domains, selected for computational diversity: dense linear algebra (gemm, gemvt), sparse operations (SpMV), reductions (asum, dot), transforms (fft_col, fft_r2), and irregular patterns (idamax). Input scales vary from embedded-typical (512-1024 elements) to HPC-relevant (up to 61K elements).

Metrics. We measure three primary metrics. *Execution cycles* (C) measure total cycles from program start to completion. *Efficiency* ($\mathcal{E}$) is defined as $\mathcal{E} = \frac{C_{ideal}}{C_{actual}} \times 100\%$, where C_{ideal} represents

Table 2. Technical comparison of scheduling strategies.

Capability	TDLS	IMS	EMS	LAEMS	FBBF
Software pipelining	✗	✓	✓	✓	✗
Non-integer effective II	✗	✗	✓	✓	✗
Dependency relaxation	✗	✗	✗	✓	✓
Store reordering	✗	✗	✗	✓	✗
Cross-boundary fusion	✗	✗	✗	✗	✓
Contribution status	Baseline	Baseline	Baseline	**Novel**	**Novel**

Table 3. Strategy configurations. Progressive configurations A–F demonstrate systematic composition from baseline to full competition. Configurations G and C–E support ablation studies. Configurations A–L support compilation time analysis.

Config	Composition	Cyclic Horizon	Linear Horizon
Progressive configurations for Section 6.2.1:			
A	TDLS(C/L)	TDLS	TDLS
B	IMS(C)+TDLS(L)	IMS	TDLS
C	EMS(C)+TDLS(L)	EMS	TDLS
D	LAEMS(C)+TDLS(L)	LAEMS	TDLS
E	LAEMS(C)+FBBF(L)	LAEMS	FBBF
F	{IMS,LAEMS,FBBF}(C)+{TDLS,FBBF}(L)	IMS, LAEMS, FBBF (compete)	TDLS, FBBF (compete)
Additional configuration for Section 6.2.2:			
G	EMS(C)+FBBF(L)	EMS	FBBF
Additional configurations for Section 6.3:			
H	FBBF(C/L)	FBBF	FBBF
I	IMS(C)+FBBF(L)	IMS	FBBF
J	{IMS,LAEMS}(C)+TDLS(L)	IMS, LAEMS (compete)	TDLS
K	LAEMS(C)+FBBF(L)+TDLS(L)	LAEMS	TDLS, FBBF (compete)
L	{IMS,LAEMS}(C)+{TDLS,FBBF}(L)	IMS, LAEMS (compete)	TDLS, FBBF (compete)
Reference baseline:			
—	Expert Assembly	Hand-optimized by assembly experts (5-8 years exp.)	

theoretical minimum cycles calculated from computational volume (total floating-point operations)[3] divided by computational resources (128 FLOPs for Matrix DSP, 96 FLOPs for MT3000-GPDSP), and C_{actual} denotes actual measured cycles. Higher efficiency indicates better instruction-level parallelism exploitation. Since C_{ideal} depends solely on program computational volume and processor resources, efficiency provides an architecture-independent metric reflecting the gap between achieved and theoretical performance. To aggregate efficiency across varied workloads without bias from outliers, we employ the geometric mean, $\mathcal{E}_{\text{geom}} = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln \mathcal{E}_i\right)$. *Compilation time* ($T_{comp}$) measures wall-clock duration (milliseconds) for scheduling, capturing practical overhead.

Strategy Configuration. Table 2 summarizes the technical distinctions among the five scheduling strategies. TDLS (list scheduling) and IMS [22] (modulo scheduling with budget-constrained backtracking) represent foundational heuristics. EMS [23] extends IMS with joint EC/II optimization for non-integer effective IIs. LAEMS and FBBF are our *novel contributions*: LAEMS adds dependency relaxation, proactive store reordering, and parallel EC search; FBBF enables cross-boundary optimization across scheduled/unscheduled regions.

We adopt expert-optimized assembly as the primary baseline following established practice in prior research [9, 11]: production VLIW backends (e.g., Hexagon) for comparable architectures remain proprietary [16, 32], academic compilers (Trimaran [33], IMPACT [34]) target the HPL-PD

[3]For matrix multiplication $A \times B$ where A has shape (M, K) and B has shape (K, N), it requires MKN multiplications and $(\mathrm{K}-1)\mathrm{MN}$ additions, totaling $(2\mathrm{K}-1)\mathrm{MN}$ FLOPs.

Table 4. Progressive multi-strategy performance across 26 instances showing systematic improvement.

Config	Composition	Efficiency (Geom. Mean)		vs. Baseline		Incremental Gain (pp)	
		Matrix	MT3000	Matrix	MT3000	Matrix	MT3000
A	TDLS(C/L)	5.1%	6.0%	—	—	—	—
B	IMS(C)+TDLS(L)	20.8%	25.2%	+308%	+320%	+15.7	+19.2
C	EMS(C)+TDLS(L)	35.9%	41.5%	+604%	+592%	+15.1	+16.3
D	LAEMS(C)+TDLS(L)	37.0%	42.9%	+625%	+615%	+1.1	+1.4
E	LAEMS(C)+FBBF(L)	37.9%	43.8%	+643%	+630%	+0.9	+0.9
F	Full Competition	37.9%	44.0%	+643%	+633%	+0.0	+0.2
—	Expert Assembly	36.6%	43.3%	+618%	+622%	—	—

architecture [35] with incompatible instruction encoding and functional unit models, and production compilers typically underperform expert assembly by 20–50% on DSP workloads [36, 37]; thus, expert code serves as the most demanding baseline.

Fairness and Correctness. All strategy configurations (A–F) operate on identical SAS inputs after code refinement (Section 3.3), ensuring observed performance differences reflect solely scheduling effectiveness. Correctness validation compares assembly execution on Matrix DSP and MT3000-GPDSP against reference C implementations with identical inputs; acceptable error margins are 10^{-5} for Matrix DSP (FP32) and 10^{-9} for MT3000-GPDSP (FP64). All 26 benchmark instances (13 programs × 2 processors) pass validation.

6.2 RQ1: Scheduling Quality and Multi-Strategy Effectiveness

This section answers three questions: How do strategies compose progressively? (Section 6.2.1) What does each algorithm contribute? (Section 6.2.2) How robust is performance across scales? (Section 6.2.3). The first two analyses use programs from Table 1 with specified input scales[4].

6.2.1 Progressive Multi-Strategy Combination. Table 4 presents the performance improvements of progressive configurations (detailed in Table 3), demonstrating systematic composition from baseline to full competition. For each configuration, we report: (1) *Geometric mean efficiency* aggregated across 13 programs; (2) *vs. Baseline*, the relative efficiency improvement compared to Config A (TDLS); (3) *Incremental gain*, the absolute efficiency increase (in percentage points, pp) from the previous configuration, showing the marginal benefit of each added strategy.

Headline Performance. Config F (full competition with {IMS, LAEMS, FBBF}(C) at cyclic horizon and {TDLS, FBBF}(L) at linear horizon) achieves geometric mean efficiency of 37.9% on Matrix DSP and 44.0% on MT3000-GPDSP, corresponding to performance ratios of **103.6% and 101.6% relative to expert assembly** by specialists with 5–8 years experience. Table 5 summarizes the performance distribution across 26 benchmark instances. In 15 instances (7+8), LASM matches or exceeds expert performance. Notable advantages include `idamax` showing 50.5%/17.7% gains through aggressive pipelining that experts avoid due to predicated control flow, and `prolongation` achieving 32.1%/32.3% advantage where LASM discovers EC=7, II=12 versus experts' conservative 4× loop unrolling.

In 11 instances with efficiency below 100% (gaps 0.1%–15.3%), primary limitations stem from greedy height-based ordering (e.g., for `asum`, experts achieve 6 vector instructions in a 6-cycle pipeline, while LASM utilizes only 5 vector instructions) and from the post-scheduling register allocator's inability to rearrange global blocks to reduce variable lifetimes (e.g., `SpMV`'s 8.1%/13.9%

[4]Input scales: gemm (M=6, K=512, N=64 for Matrix DSP, N=48 for MT3000-GPDSP), gemvt (M=512, N=64), asum/scal (N=61440), axpby (N=32768), dot (N=30720), fft_col (N=512), fft_r2 (N=1024), SpMV (nir=1728, lr=27), coeff_update/mul_coeff (N=512), prolongation (lr=8192), idamax (N=49152).

Table 5. Config F performance versus expert assembly across 26 instances. Performance ratio: $C_{\text{expert}}/C_{\text{LASM}} \times 100\%$.

Metric	Matrix DSP	MT3000-GPDSP
Performance ratio (LASM relative to Expert):		
Arithmetic mean	104.7%	102.3%
Geometric mean	103.6%	101.6%
Standard deviation	16.7%	12.2%
Distribution:		
Instances $\geq$ 100%	7/13 (53.8%)	8/13 (61.5%)
Maximum advantage	+50.5% (idamax)	+32.3% (prolongation)
Maximum shortfall	−15.3% (asum)	−13.9% (SpMV)
Absolute efficiency (relative to theoretical optimum):		
Config F (geom. mean)	37.9%	44.0%
Expert (geom. mean)	36.6%	43.3%

gaps). These gaps highlight future opportunities while demonstrating LASM achieves competitive performance with substantially reduced manual effort.

Progressive Composition. Table 4 demonstrates systematic improvement through strategy addition:

Baseline (Config A). TDLS establishes 5.1%/6.0% efficiency baseline, handling basic blocks adequately but lacking loop optimization, manifesting as 3.2×–35.0× gaps from theoretical optimum on loop-dominated benchmarks.

Software pipelining (Config B). IMS achieves 20.8%/25.2% efficiency, demonstrating specialized horizons address distinct optimization challenges. Compared to baseline, this shows +308%/+320% relative improvement with +15.7/+19.2 percentage points incremental gain. Dramatic improvement stems from loop pipelining reducing execution time by 3.66× geometric mean. But IMS's integer II limits performance where non-integer effective IIs would benefit.

Expanded search (Config C). EMS reaches 35.9%/41.5% (+604%/+592% vs. baseline, +15.1/+16.3 pp incremental gain) by exploring multiple (EC, II) combinations. This represents the second major leap, closing gap left by IMS's integer constraint through systematic exploration of unrolling factors.

Algorithmic innovations (Config D). LAEMS achieves 37.0%/42.9% (+625%/+615% vs. baseline, +1.1/+1.4 pp incremental gain), reaching 101.1%/99.1% of expert performance. The modest incremental gain reflects that EMS already explores broad search space; LAEMS's innovations (dependency relaxation, store reordering) target specific challenging patterns where EMS fails. On MT3000-GPDSP, the 1.4 percentage point gain demonstrates LAEMS's effectiveness across more benchmarks (6/13 vs 3/13 on Matrix DSP).

Cross-boundary fusion (Config E). Adding FBBF reaches 37.9%/43.8% (+643%/+630% vs. baseline, +0.9/+0.9 pp incremental gain), achieving 103.6%/101.2% of expert performance. The consistent incremental gain demonstrates that cross-boundary fusion fills delay slots across diverse patterns.

Full competition (Config F). Full competition achieves 37.9%/44.0% (+643%/+633% vs. baseline, +0.0/+0.2 pp incremental gain), showing marginal improvement over Config E. Analysis reveals IMS selection in only 2/13 benchmarks (`fft_col`, `fft_r2`), where simpler integer II proves sufficient. FBBF(C) registration provides minimal additional benefit (7 cycles average in 2/13 cases). However, full competition represents the most robust configuration, making no workload assumptions and automatically discovering optimal strategy combinations per code region.

6.2.2 Algorithmic Contributions via Ablation Study. We employ a 2×2 factorial design [38] to isolate: (1) *LAEMS main effect*: Performance gain from algorithmic innovations (store reordering, dependency relaxation) over EMS; (2) *FBBF main effect*: Performance gain from cross-boundary

Table 6. LAEMS algorithmic contribution comparing EMS(C)+TDLS(L) vs. LAEMS(C)+TDLS(L). $\mathcal{C}$(execution cycles); $\mathcal{E}$(efficiency).

Benchmark	Matrix DSP (FP32)				MT3000-GPDSP (FP64)			
	EMS $\mathcal{C}$	LAEMS $\mathcal{C}$	EMS $\mathcal{E}$	LAEMS $\mathcal{E}$	EMS $\mathcal{C}$	LAEMS $\mathcal{C}$	EMS $\mathcal{E}$	LAEMS $\mathcal{E}$
gemm	3137	3137	97.9%	97.9%	3128	3128	98.2%	98.2%
gemvt	620	620	83.9%	83.9%	843	792	81.1%	86.4%
asum	2453	2453	39.1%	39.1%	3924	3924	32.6%	32.6%
axpby	1709	1581	44.9%	48.6%	2087	1826	49.1%	56.1%
scal	1945	1945	24.7%	24.7%	1943	1943	32.9%	32.9%
dot	1095	1095	43.8%	43.8%	1215	1215	52.7%	52.7%
fft_col	16201	16201	35.6%	35.6%	20285	16534	37.9%	46.4%
fft_r2	2010	1637	19.9%	24.4%	1410	1397	37.9%	38.2%
SpMV	2718	2718	26.8%	26.8%	3786	3786	25.7%	25.7%
coeff_update	1442	1442	53.3%	53.3%	1686	1650	60.7%	62.1%
mul_coeff	2081	1828	36.9%	42.0%	1567	1567	65.3%	65.3%
prolongation	416	416	15.4%	15.4%	417	415	20.6%	20.7%
idamax	4220	4220	18.2%	18.2%	6288	6288	16.3%	16.3%
Geom. mean	—	—	35.9%	37.0%	—	—	41.5%	42.9%

fusion over TDLS; (3) *Interaction effect*: Whether combining strategies yields multiplicative benefits. Two factors vary independently (Factor 1: EMS vs. LAEMS; Factor 2: TDLS vs. FBBF), yielding four treatment combinations.

Main effect of LAEMS. Table 6 compares EMS and LAEMS across 13 benchmarks, holding linear strategy constant at TDLS. LAEMS achieves geometric mean efficiency of 37.0% compared to EMS's 35.9% on Matrix DSP, and 42.9% compared to 41.5% on MT3000-GPDSP. This represents overall relative improvements of 3.1% and 3.4%, respectively.

While LAEMS maintains identical schedules for 10/13 programs on Matrix DSP and 7/13 programs on MT3000-GPDSP, it demonstrates improvements in challenging patterns. Two mechanisms drive these improvements:

Proactive store reordering prevents ordering violations and enables lower initiation intervals. On Matrix DSP, `axpby` achieves 48.6% efficiency versus EMS's 44.9% (+8.2%, relative efficiency improvement): EMS fails at II=12 when EC=7 due to store ordering conflicts, forcing II=13 (1709 cycles), while LAEMS preemptively detects and resolves conflicts, achieving II=12 (1581 cycles). Similarly, `mul_coeff` gains 13.8% efficiency (42.0% vs. 36.9%) through LAEMS's EC=1, II=7 versus EMS's EC=1, II=8. For `fft_r2` containing six nested loops, LAEMS achieves 22.6% efficiency improvement (24.4% vs. 19.9%) by producing lower II values in three loops through proactive reordering. On MT3000-GPDSP, `axpby` demonstrates the most dramatic gain (+14.3%): EMS cannot successfully schedule at EC=7 due to store conflicts and falls back to EC=0, while LAEMS successfully pipelines at EC=7. For `fft_col` with three nested loops (+22.4%), LAEMS achieves EC=0, II=6 for innermost loops while EMS requires II=8.

Selective dependency relaxation enables aggressive pipelining through strategic move instruction insertion. On MT3000-GPDSP, `gemvt` improves 6.4% efficiency (86.4% vs. 81.1%) as LAEMS discovers EC=7 versus EMS's conservative EC=3. For `coeff_update` (+2.2%), LAEMS achieves EC=6, II=21 versus EMS's EC=7, II=24. For `prolongation` (+0.5%), LAEMS reaches EC=7, II=13 versus EMS's EC=1, II=3. The modest percentage gains in these cases reflect that the programs already operate at moderate efficiency; even small absolute improvements represent meaningful optimization.

Main effect of FBBF. Table 7 compares FBBF against TDLS, holding cyclic strategy constant at LAEMS. FBBF achieves geometric mean efficiency of 37.9% compared to TDLS's 37.0% on Matrix

Table 7. FBBF cross-boundary fusion contribution comparing LAEMS(C)+TDLS(L) vs. LAEMS(C)+FBBF(L). $\mathcal{C}$(execution cycles); $\mathcal{E}$(efficiency).

Benchmark	Matrix DSP (FP32)				MT3000-GPDSP (FP64)			
	TDLS $\mathcal{C}$	FBBF $\mathcal{C}$	TDLS $\mathcal{E}$	FBBF $\mathcal{E}$	TDLS $\mathcal{C}$	FBBF $\mathcal{C}$	TDLS $\mathcal{E}$	FBBF $\mathcal{E}$
gemm	3137	3110	97.9%	98.8%	3128	3101	98.2%	99.1%
gemvt	620	605	83.9%	86.0%	792	773	86.4%	88.5%
asum	2453	2428	39.1%	39.5%	3924	3909	32.6%	32.7%
axpby	1581	1566	48.6%	49.0%	1826	1815	56.1%	56.4%
scal	1945	1938	24.7%	24.8%	1943	1936	32.9%	33.1%
dot	1095	1075	43.8%	44.7%	1215	1207	52.7%	53.0%
fft_col	16201	15577	35.6%	37.0%	16534	15821	46.4%	48.5%
fft_r2	1637	1499	24.4%	26.7%	1397	1273	38.2%	41.9%
SpMV	2718	2581	26.8%	28.2%	3786	3605	25.7%	27.0%
coeff_update	1442	1435	53.3%	53.5%	1650	1638	62.1%	62.5%
mul_coeff	1828	1823	42.0%	42.1%	1567	1565	65.3%	65.4%
prolongation	416	405	15.4%	15.8%	415	407	20.7%	21.1%
idamax	4220	4188	18.2%	18.3%	6288	6254	16.3%	16.4%
Geom. mean	—	—	37.0%	37.9%	—	—	42.9%	43.8%

DSP, and 43.8% compared to 42.9% on MT3000-GPDSP. This represents overall relative efficiency improvements of 2.4% and 2.1%, respectively.

Cross-boundary fusion achieves improvements on all 13/13 programs on both platforms, demonstrating systematic benefits across diverse patterns. FBBF fills delay slots by moving operations from unscheduled blocks into scheduled kernels: `gemm` fuses 3 address computations (-27 cycles); `SpMV`'s irregular access creates 5-6 fusible index calculations (+5.3% efficiency improvement on Matrix DSP, +5.0% on MT3000-GPDSP); largest gains appear in `fft_r2` (+9.2% on Matrix DSP, +9.7% on MT3000-GPDSP), which is achieved by effectively utilizing the idle delay slots in the prologue and epilogue of its three pipelined loops. Even programs showing modest absolute improvements (0.1%-1.0% efficiency gain) benefit from FBBF's opportunistic delay slot filling, validating the universal applicability of cross-boundary fusion.

Strategy synergy. To assess whether LAEMS and FBBF produce multiplicative (synergistic) or additive (independent) effects, we compute the interaction metric:

$$\Delta = (C_{\text{LAEMS+TDLS}} - C_{\text{LAEMS+FBBF}}) - (C_{\text{EMS+TDLS}} - C_{\text{EMS+FBBF}}) \tag{14}$$

where $\Delta = 0$ indicates pure additivity, while $\Delta \neq 0$ suggests interaction effects. Analysis reveals predominantly additive behavior: 12/13 benchmarks on Matrix DSP and 8/13 on MT3000-GPDSP show $|\Delta| < 5$ cycles (<0.5% of total execution). Non-zero cases exhibit minimal interaction ($|\Delta| < 10$ cycles). This independence simplifies framework design: strategies can be developed and composed without concern for complex cross-strategy dependencies.

6.2.3 Performance Convergence Across Input Scales. We further select programs from Table 1 to evaluate LASM's optimization performance under different input scales. Figure 10 demonstrates convergence patterns as input scales vary from small to large across representative configurations (Config A,B,F).

Convergence pattern. For most benchmarks, efficiency increases monotonically with scale, plateauing when iterations dominate. Larger scales enhance loop optimization while amortizing fixed overheads (prologue, epilogue). Matrix multiplication (M=6, N=48, varying K) on MT3000-GPDSP shows LASM-expert gap shrinking from 0.2% (K=64) to near parity (K=512), where both achieve 99.1% efficiency. Fixed costs (prologue, epilogue) amortize from 13.2% at K=64 to 3.1% at K=512, validating LASM's suitability for production HPC workloads. Similar convergence appears

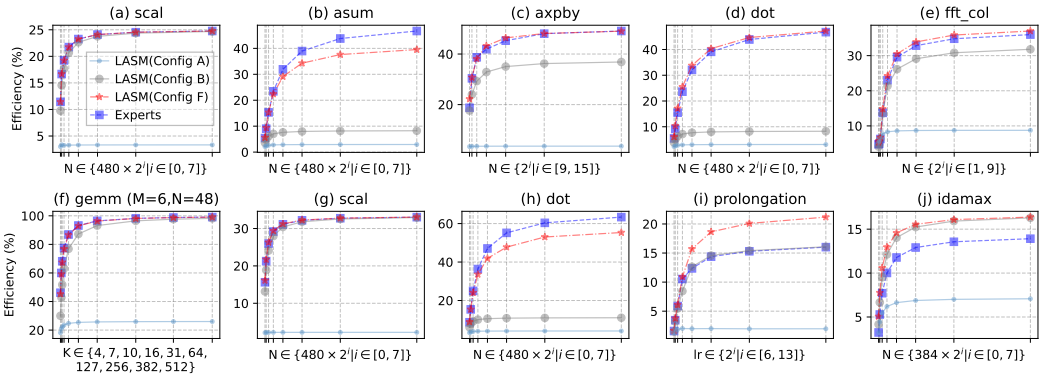


Fig. 10. Benchmark efficiency under input scales from small to large. The upper subplots present results on Matrix DSP and the lower subplots present results on MT3000-GPDSP.

across platforms: on Matrix DSP, `scal` and `dot` show LASM matching or slightly exceeding expert performance at large scales (N=61440) while maintaining competitive efficiency at small scales (N=480).

LASM advantages at scale. `prolongation` demonstrates LASM outperforming experts for lr $\geq$ 512 on MT3000-GPDSP. Experts achieve a loop pipeline with 4 unrolls and II=6, while LASM achieves a pipeline with 7 unrolls and II=12. For lr=8192, prologue becomes negligible (<1%), and superior effective II dominates: 24% lower cycles (407 vs. 537), demonstrating 32.3% LASM advantage in Table 5. Crossover at lr$\approx$512 illustrates systematic exploration automatically discovering scale-dependent optimal configurations. Similarly, `idamax` shows increasing LASM advantage with scale: from near parity at N=384 to 50.5% advantage at N=49152 on Matrix DSP.

Expert advantages at scale. `asum` on Matrix DSP shows widening gaps as N increases: near parity at N=480 (5.6% LASM vs. 5.4% expert) widens to 7.2% gap at N=61440 (39.5% LASM vs. 46.7% expert). Experts manually orchestrate instruction scheduling to achieve superior instruction packing, maintaining 6 vector instructions within a 6-cycle pipeline through careful dependency management. In contrast, the greedy height-based heuristic (employed in LAEMS strategy) produces suboptimal instruction orderings for `asum`'s reduction pattern, achieving only 5 vector instructions in the 6-cycle pipeline. For N=61440, repeated pipelined kernel amplifies the per-iteration inefficiency, accumulating to 372 cycles overhead (7.2% efficiency gap). Similar patterns appear on MT3000-GPDSP `dot`: experts maintain 63.4% efficiency at N=61440 versus LASM's 55.3% (8.1% gap), attributed to superior instruction ordering that achieves better functional unit utilization through manual scheduling optimization beyond the greedy heuristic's scope.

Three findings emerge: (1) *Large-scale validation*: LASM achieves 99.1% expert efficiency at gemm K=512, validating production HPC suitability; (2) *Small-scale opportunities*: 2–7% gaps suggest specialized strategies prioritizing prologue/epilogue efficiency; (3) *Crossover insights*: Scale-dependent discoveries could inform auto-tuning with profiling-guided configuration selection.

6.3 RQ2: Compilation Time Analysis

Figure 11 presents compilation times across 12 configurations on 26 benchmark instances. We use the input scales as specified in Section 6.2 (see footnote) to ensure consistent comparison. Each configuration is compiled with 3 warm-up runs followed by 10 measured runs; reported values represent means across these runs and across the 13 programs per platform.

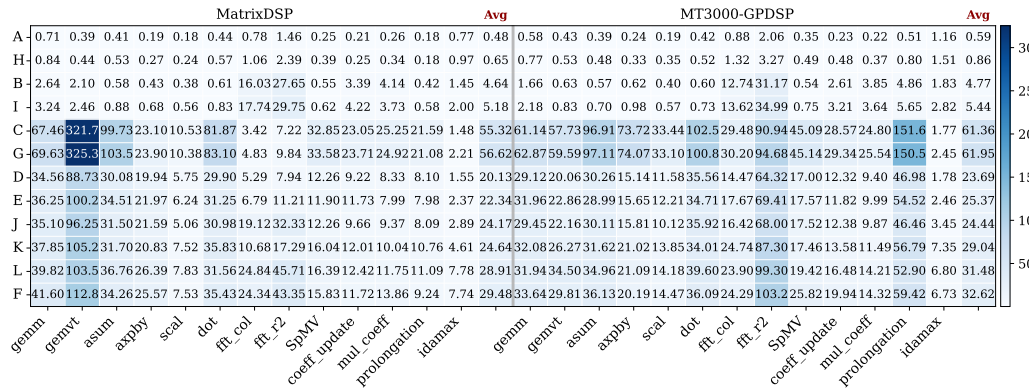


Fig. 11. Compilation times (milliseconds) across configurations A–L (Table 3) for 26 benchmark instances. Each cell shows wall-clock compilation time for a program-configuration pair. The last column shows the average compilation time for each strategy configuration. Color intensity indicates compilation time magnitude, ranging from 0.18 ms (TDLS on `scal`) to 325.35 ms (EMS on `gemvt`). LAEMS (Config D) consistently achieves lower compilation times than EMS (Config C) while maintaining scheduling quality. Full competition (Config F) shows modest overhead versus single-strategy configurations.

LAEMS algorithmic efficiency. EMS (Config C) explores all expansion configurations EC $\in [0, 8]$ sequentially, incurring 115.25× and 104× overhead versus baseline TDLS, with mean compilation times of 55.32 ms and 61.36 ms on Matrix DSP and MT3000-GPDSP respectively. LAEMS (Config D) achieves 2.75× and 2.59× speedup over EMS with mean times of 20.13 ms and 23.69 ms. This efficiency stems from three algorithmic mechanisms: proactive store conflict detection eliminates redundant backtracking (∼1.4× contribution), early termination upon finding feasible (EC, II) pairs prevents exhaustive search (∼1.6× contribution), and selective dependency relaxation reduces exploration space (∼1.2× contribution). Combined multiplicatively, these yield the observed 2.7× speedup. Search space analysis confirms efficiency: EMS explores 9 EC values exhaustively, while LAEMS's speedup suggests it examines only ∼3.3 values on average before convergence (a 63% reduction). Programs with complex memory dependencies benefit most: `gemvt` achieves 3.63× faster compilation on Matrix DSP (321.7 ms → 88.7 ms), while `axpby` gains 4.88× on MT3000-GPDSP (73.7 ms → 15.1 ms). Beyond speed, LAEMS exhibits 25.5% better compilation time predictability (coefficient of variation 1.156 versus EMS's 1.551 on Matrix DSP), indicating more consistent behavior across diverse program patterns.

Framework parallel scheduling efficiency. The framework executes cyclic strategies in parallel for same-level loop nests (Section 4.1). Config J evaluates this by adding IMS to LAEMS at the cyclic horizon. We estimate the isolated IMS overhead as 4.26 ms (Matrix DSP) and 4.28 ms (MT3000-GPDSP) by subtracting baseline Config A time from Config B time[5]. Sequential execution would require at least 24.39 ms and 27.97 ms (Config D plus IMS overhead), while parallel execution achieves 24.17 ms and 24.44 ms respectively, yielding efficiencies of 1.01× and 1.14×. These results reflect workload characteristics. LAEMS exhibits 4.63× (Matrix DSP) and 5.42× (MT3000-GPDSP) higher overhead than IMS, creating severe load imbalance where LAEMS dominates as the critical path. Under such imbalance, theoretical speedup is bounded by $\frac{T_{\text{LAEMS}}+T_{\text{IMS}}}{\max(T_{\text{LAEMS}},T_{\text{IMS}})} \approx 1.22\times$ and 1.18×, respectively. MT3000-GPDSP's superior 14% speedup demonstrates effective exploitation of our 12-core experimental environment, while Matrix DSP encounters greater compilation infrastructure

[5]Lower bound estimate: assumes TDLS(C) overhead ≈ 0.1. Actual TDLS(C)+TDLS(L) (Config A) overhead is 0.48 ms (Matrix DSP) and 0.59 ms (MT3000-GPDSP), making the TDLS(C) overhead negligible.

Table 8. Code volume reduction through LAN across 13 benchmarks on both platforms.

Program	Matrix DSP			MT3000-GPDSP		
	LAN	Assembly	Reduction	LAN	Assembly	Reduction
gemm	147	271	45.8%	120	236	49.2%
gemvt	64	327	80.4%	49	324	84.9%
asum	70	276	74.6%	66	218	69.7%
axpby	31	213	85.4%	32	215	85.1%
scal	29	166	82.5%	27	137	80.3%
dot	77	279	72.4%	93	261	64.4%
fft_col	171	518	67.0%	158	643	75.4%
fft_r2	282	1113	74.7%	271	1296	79.1%
SpMV	58	195	70.3%	50	185	73.0%
coeff_update	36	233	84.5%	29	314	90.8%
mul_coeff	38	186	79.6%	29	236	87.7%
prolongation	24	148	83.8%	67	320	79.1%
idamax	184	279	34.1%	242	330	26.7%
Total	1211	4204	71.19%	1233	4715	73.85%

contention. Both platforms avoid naive sequential overhead, validating the parallel orchestration design.

Critically, the current two-strategy scenario represents the most challenging case for parallelization. As LASM integrates additional strategies with more balanced computational costs, the framework's parallel mechanism will demonstrate substantially greater advantages through improved load distribution across cores. The infrastructure successfully proves its correctness under worst-case imbalance; future multi-strategy configurations will naturally unlock its full potential.

Competition overhead and practical implications. Full competition (Config F) with 5 strategies achieves 29.48 ms and 32.62 ms mean compilation times, representing 32.0% and 28.6% overhead versus Config E (single-strategy per horizon at 22.34 ms and 25.37 ms). Three factors render this overhead acceptable: (1) the additional ~7 ms compilation time remains negligible relative to typical development cycles and program execution times (even worst-case `gemvt` completes in under 150 ms); (2) overhead scales sub-linearly with strategy count due to parallel execution and shared infrastructure—adding IMS to Config E incurs only 7.14 ms/7.25 ms despite IMS requiring 4.26 ms/4.28 ms in isolation; (3) full competition provides robustness by eliminating workload-specific configuration—IMS wins in 2/13 benchmarks (`fft_col`, `fft_r2`) where simpler integer II proves optimal, patterns difficult to predict *a priori*. For production deployment, Config E (LAEMS+FBBF) balances efficiency and performance. Config D (LAEMS+TDLS) offers faster compilation (20.13 ms, 23.69 ms) suitable for development iterations. Config F justifies its overhead for critical applications requiring maximum robustness through strategy diversity.

6.4 RQ3: Engineering Cost Analysis

We quantify engineering cost through lines of code (LOC) written and strategy integration effort.

Source Lines of Code. Table 8 presents LAN versus manual assembly across 13 benchmarks, with all line counts excluding blank lines. LAN totals 2,444 LOC compared to 8,919 LOC in manual assembly, achieving 72.60% overall reduction (71.19% on Matrix DSP, 73.85% on MT3000-GPDSP). Reduction stems from eliminating explicit parallelism markers, replacing manual stack management with pseudo-operations, and using symbolic variables instead of explicit register allocation. Individual reductions range from 26.7% to 90.8%, with variation reflecting the proportion of scheduling overhead in manual assembly.

Strategy Integration Effort. The five evaluated strategies require approximately 30 lines of interface code for registration at appropriate horizons and command-line option declarations.

Table 9. YOLOv5 deployment comparison on Matrix DSP (single core) at 1 GHz processing 640 × 640 images.

Metric	Expert Assembly	LASM
Execution cycles	187,962,440	196,051,920
Latency (ms)	188.0	196.1
Performance ratio	100%	95.9%
Development effort*	9 engineer-weeks	3 engineer-weeks
Compilation time	N/A	129.2 ms

* Measures implementation time: 9 weeks for manual assembly optimization, 3 weeks for writing 1469 LOC LAN (excluding blank lines). Excludes algorithm development time.

Strategy implementations use unified scheduling primitives, with development time averaging 2–5 days per strategy. This lightweight extensibility enables rapid integration and evaluation of novel scheduling techniques.

6.5 RQ4: Real-World Application Deployment

Prior assembly optimization frameworks [18, 39] target isolated kernels, unable to handle complete programs with function calls and complex control flow. Here, we validate LASM's production applicability by deploying YOLOv5-6.0s model [40] on Matrix DSP single core. The model comprises 60 convolutional layers, 3 pooling layers, and 2 upsampling layers. Implementation in LAN spans 1469 LOC (excluding blank lines), utilizing four customized GEMM kernel sizes $(1, K, 64)$, (6,K,64), (1,K,32), (6,K,32) with $K \in [1, 384]$. Since the instruction set of Matrix DSP lacks exponential vector operations, we use LeakyReLU instead of the SiLU activation function.

Compilation with the configuration LAEMS(C)+FBBF(L) completes in 129.2 ms. Table 9 shows LASM achieves 95.9% of expert performance while enabling a 3× productivity improvement under equivalent algorithmic requirements. The 4.3% performance gap primarily stems from expert assembly's advantages in global cross-layer optimization. Our extensible framework enables integration of advanced global optimization strategies to further narrow this gap. Validation on 100 COCO images yields mean absolute error $< 10^{-4}$ versus PyTorch reference.

The 65-layer architecture offers significant optimization space, which experts exploit through manual global coordination, while LASM's systematic approach enables rapid strategy integration. This deployment validates LASM's capability for production applications beyond isolated kernels, handling multi-layer dependencies and complete workflows with performance approaching expert assembly while substantially reducing development effort.

7 Related Work

LASM addresses a critical gap in VLIW optimization: assembly scheduling with competitive multi-strategy orchestration for complete programs. We position our work relative to three research areas: instruction scheduling algorithms, compiler frameworks, and DSP kernel optimization.

Instruction scheduling algorithms. Software pipelining dominates VLIW loop optimization. Lam [41] introduced modulo scheduling's foundational framework with initiation interval minimization. Rau's Iterative Modulo Scheduling (IMS) [22] made the approach practical through budget-constrained backtracking, earning MICRO's Test of Time Award. Llosa et al.'s Swing Modulo Scheduling (SMS) [25] advanced the state-of-the-art through lifetime-sensitive operation placement, achieving near-optimal register allocation while maintaining minimal stage counts; SMS remains the primary software pipelining technique in GCC [15] and LLVM [14] through 2025. Expanded Modulo Scheduling (EMS) [23] explores non-integer effective initiation intervals through adaptive

loop expansion. For acyclic code, list scheduling provides the foundational heuristic, while trace scheduling [42], superblock scheduling [43], and hyperblock scheduling [44] extend optimization across basic block boundaries. Recent work includes formally verified schedulers [19, 45] and energy-aware approaches [46]. *LASM distinguishes itself through competitive scheduling where multiple strategies simultaneously optimize identical code regions, with the framework automatically selecting the best implementable solution through two-tier feasibility filtering: a capability absent from fixed-strategy approaches.*

VLIW compiler frameworks. LLVM [14] dominates modern VLIW development through DFAPacketizer for instruction bundling, with Qualcomm's Hexagon [16] representing the most mature backend. Recent work by Kalray [17] reports substantial effort required for VLIW backends, identifying scheduling as the hardest part with extensive modifications to MachinePipeliner, DFAPacketizer, and VLIWPacketizerList. GCC maintains SMS [25] for software pipelining and DFA-based instruction scheduling. Trimaran [33] provided retargetable VLIW compilation for ILP research but lacks mechanisms for multi-strategy composition. High-level compiler frameworks face inherent limitations: phase ordering problems between scheduling and register allocation, conservative resource modeling, and information loss across IR transformations. *LASM operates at the symbolic assembly level, scheduling actual ISA instructions with symbolic operands and precise hardware knowledge, avoiding these abstraction penalties.*

Critically, existing compiler frameworks commit to fixed instruction scheduling strategies: LLVM employs Swing Modulo Scheduling through MachinePipeliner for loops and greedy packetization via DFAPacketizer for bundling, while GCC maintains its own modulo scheduling implementation. No existing framework provides mechanisms for multiple scheduling algorithms to simultaneously optimize identical code regions with automatic best-solution selection. *LASM introduces this capability through competitive multi-strategy orchestration, where multiple strategies generate candidate schedules that compete through parallel exploration with feasibility-driven selection.*

DSP kernel optimization. Texas Instruments' Linear Assembly Optimizer [18] for TMS320C6x DSPs represents sophisticated production-quality assembly optimization, accepting symbolic registers and producing optimized parallel assembly through integrated register allocation and modulo scheduling. However, it employs a single fixed scheduling strategy and focuses on loop kernels without support for complete programs. SALTO [39] provides a retargetable framework for assembly profiling and optimization but operates on fully scheduled assembly with limited VLIW-specific support. JOKer [47] takes a different approach through end-to-end kernel compilation: it integrates loop transformation, vectorization, and instruction scheduling within a unified reinforcement learning framework, achieving impressive results on isolated DSP kernels. *LASM differs from these approaches through its extensible multi-strategy framework that enables systematic comparison and composition of diverse scheduling algorithms, combined with support for complete programs beyond isolated kernels.*

8 Conclusion and Future Work

This article presents LASM, an extensible symbolic assembly framework for competitive multi-strategy orchestration of complete VLIW programs beyond isolated kernels. LASM addresses the performance-productivity gap in VLIW development through three design elements: standardized extension interfaces for strategy integration, unified scheduling primitives that reduce algorithmic duplication, and two-tier feasibility filtering that selects an implementable schedule from competing candidates. The framework supports diverse optimization strategies without committing to a single scheduling algorithm. This work demonstrates the design through two representative strategies: LAEMS for aggressive loop pipelining with dependency relaxation and proactive store reordering, and FBBF for cross-boundary optimization. Evaluation on 13 benchmarks across two production

VLIW-SIMD processors shows that LASM achieves 103.6% and 101.6% of expert assembly performance while reducing manual code by 72.6%. A full YOLOv5 deployment further demonstrates production applicability at 95.9% of expert performance. These results show that symbolic assembly can preserve architecture-level control while reducing manual scheduling effort.

Future work uses the strategy-performance data that competitive evaluation generates to study learning-guided strategy selection. It also extends LASM to emerging VLIW architectures and investigates tighter integration with high-level frameworks for end-to-end compilation workflows.

Acknowledgments

This work is supported in part by the Innovation Research Foundation of NUDT under Grant Nos. 23-ZZCX-JDZ-11 and 25-ZZCX-JDZ-16, and by the National Natural Science Foundation of China under Grant Nos. 62472435 and 62421002.

References

[1] Joseph A. Fisher, Paolo Faraboschi, and Cliff Young. 2005. *Embedded computing - a VLIW approach to architecture, compilers, and tools.* Morgan Kaufmann, San Francisco, CA, USA.

[2] Lucian Codrescu, Willie Anderson, Suresh Venkumanhanti, Mao Zeng, Erich Plondke, Chris Koob, Ajay Ingle, Charles Tabony, and Rick Maule. 2014. Hexagon DSP: An Architecture Optimized for Mobile Multimedia and Communications. *IEEE Micro* 34, 2 (2014), 34–43. doi:10.1109/MM.2014.12

[3] Texas Instruments. 2020. C7000 C/C++ Optimization User's Guide. https://www.ti.com/lit/ug/spruiv4b/spruiv4b.pdf

[4] Texas Instruments. 2021. TDA4VM Jacinto Processors for ADAS and Autonomous Vehicles. https://www.ti.com/lit/ds/symlink/tda4vm.pdf

[5] AMD Xilinx. 2025. AMD Versal AI Engine Architecture. https://docs.amd.com/r/en-US/ug1079-ai-engine-kernel-coding. https://docs.amd.com/r/en-US/ug1079-ai-engine-kernel-coding Technical documentation.

[6] Intel Corp. 2023. Intel Gaudi2 AI Accelerators. https://habana.ai/products/gaudi2

[7] Kai Lu, Yaohua Wang, Yang Guo, Chun Huang, Sheng Liu, Ruibo Wang, Jianbin Fang, Tao Tang, Zhaoyun Chen, Biwei Liu, Zhong Liu, Yuanwu Lei, and Haiyan Sun. 2022. MT-3000: a heterogeneous multi-zone processor for HPC. *CCF Trans. High Perform. Comput.* 4, 2 (2022), 150–164. doi:10.1007/S42514-022-00095-Y

[8] Shuming Chen, Sheng Liu, Jianghua Wan, Yaohua Wang, Shenggang Chen, Haiyan Chen, Hengzhu Liu, Haiyan Sun, and Zhong Liu. 2015. Coordinate multi-core DSP YHFT-QMBase: architecture and implementation. *SCIENTIA SINICA Informationis* 45, 4 (2015), 560–573. doi:10.1360/n112014-00298

[9] Murtaza Ali, Eric Stotzer, Francisco D. Igual, and Robert A. van de Geijn. 2012. Level-3 BLAS on the TI C6678 Multi-core DSP. In *IEEE 24th International Symposium on Computer Architecture and High Performance Computing, SBAC-PAD 2012, New York, NY, USA, October 24-26, 2012*. IEEE Computer Society, Los Alamitos, CA, USA, 179–186. doi:10.1109/SBAC-PAD.2012.26

[10] Mounir Bahtat, Said Belkouch, Philippe Elleaume, and Philippe Le Gall. 2016. Instruction scheduling heuristic for an efficient FFT in VLIW processors with balanced resource usage. *EURASIP Journal on Advances in Signal Processing* 2016, 1 (2016), 38. doi:10.1186/s13634-016-0336-0

[11] Gianluca Frison, Dimitris Kouzoupis, Tommaso Sartor, Andrea Zanelli, and Moritz Diehl. 2018. BLASFEO: Basic Linear Algebra Subroutines for Embedded Optimization. *ACM Trans. Math. Softw.* 44, 4, Article 42 (July 2018), 30 pages. doi:10.1145/3210754

[12] Can Deng, Zhaoyun Chen, Yang Shi, Yimin Ma, Mei Wen, and Lei Luo. 2024. Optimizing VLIW Instruction Scheduling via a Two-Dimensional Constrained Dynamic Programming. *ACM Trans. Des. Autom. Electron. Syst.* 29, 5, Article 83 (Sept. 2024), 20 pages. doi:10.1145/3643135

[13] J. D. Ullman. 1975. NP-complete scheduling problems. *J. Comput. Syst. Sci.* 10, 3 (June 1975), 384–393. doi:10.1016/S0022-0000(75)80008-0

[14] Chris Lattner and Vikram S. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *2nd IEEE / ACM International Symposium on Code Generation and Optimization (CGO 2004), 20-24 March 2004, San Jose, CA, USA*. IEEE Computer Society, Los Alamitos, CA, USA, 75–88. doi:10.1109/CGO.2004.1281665

[15] Diego Novillo. 2006. GCC: An Architectural Overview, Current Status, and Future Directions. Proceedings of the Linux Symposium, Volume Two, pp. 185–200, Ottawa, Ontario, Canada. https://www.kernel.org/doc/ols/2006/ols2006v2-pages-193-208.pdf

[16] Bob Simpson. 2011. Porting LLVM to a Next Generation DSP. Presented at the LLVM Developers' Meeting. Available at: https://www.llvm.org/devmtg/2011-11/.

[17] Kalray SA. 2022. Developing an LLVM Backend for the KV3 Kalray VLIW Core. Presented at EuroLLVM 2022, Online. https://llvm.org/devmtg/2022-05/slides/2022EuroLLVM-DevelopingAnLLVMBackend-for-the-KV3KalrayVLIWCore.pdf

[18] Texas Instruments. 2015. TMS320C6000 Optimizing C/C++ Compiler v8.3.x User's Guide. https://www.ti.com/lit/ug/sprui04f/sprui04f.pdf

[19] Cyril Six, Sylvain Boulmé, and David Monniaux. 2020. Certified and efficient instruction scheduling: application to interlocked VLIW processors. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 129 (Nov. 2020), 29 pages. doi:10.1145/3428197

[20] Jonathan Ragan-Kelley, Andrew Adams, Dillon Sharlet, Connelly Barnes, Sylvain Paris, Marc Levoy, Saman Amarasinghe, and Frédo Durand. 2017. Halide: decoupling algorithms from schedules for high-performance image processing. *Commun. ACM* 61, 1 (Dec. 2017), 106–115. doi:10.1145/3150211

[21] Dave Estes. 2014. Machine Instruction Scheduler Tutorial. LLVM Developer Meeting Slides. https://llvm.org/devmtg/2014-10/Slides/Estes-MISchedulerTutorial.pdf Presented at LLVM Dev Meeting, October 2014.

[22] B. Ramakrishna Rau. 1994. Iterative modulo scheduling: an algorithm for software pipelining loops. In *Proceedings of the 27th Annual International Symposium on Microarchitecture, San Jose, California, USA, November 30 - December 2, 1994*. ACM / IEEE Computer Society, New York, NY, USA / Los Alamitos, CA, USA, 63–74. doi:10.1109/MICRO.1994.717412

[23] Hongli Zhong, Zhong Liu, Sheng Liu, Sheng Ma, and Chen Li. 2022. Adaptive Low-Cost Loop Expansion for Modulo Scheduling. In *Network and Parallel Computing - 19th IFIP WG 10.3 International Conference, NPC 2022, Jinan, China, September 24-25, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13615)*. Springer, Cham, Switzerland, 30–41. doi:10.1007/978-3-031-21395-3_3

[24] Christoph W. Kessler. 2013. Compiling for VLIW DSPs. In *Handbook of Signal Processing Systems*, Shuvra S. Bhattacharyya, Ed F. Deprettere, Rainer Leupers, and Jarmo Takala (Eds.). Springer, New York, NY, USA, 1177–1214. doi:10.1007/978-1-4614-6859-2_36

[25] Josep Llosa, Eduard Ayguadé, Antonio González, Mateo Valero, and Jason Eckhardt. 2001. Lifetime-Sensitive Modulo Scheduling in a Production Environment. *IEEE Trans. Computers* 50, 3 (2001), 234–249. doi:10.1109/12.910814

[26] Ivan Llopard, Albert Cohen, Christian Fabre, Jérôme Martin, Henri-Pierre Charles, and Christian Bernard. 2013. Code generation for an application-specific VLIW processor with clustered, addressable register files. In *Proceedings of the 10th Workshop on Optimizations for DSP and Embedded Systems (ODES '13)*. Association for Computing Machinery, New York, NY, USA, 11–19. doi:10.1145/2443608.2443612

[27] Roel Jordans and Henk Corporaal. 2015. High-level software-pipelining in LLVM. In *Proceedings of the 18th International Workshop on Software and Compilers for Embedded Systems* (Sankt Goar, Germany) *(SCOPES '15)*. Association for Computing Machinery, New York, NY, USA, 97–100. doi:10.1145/2764967.2771935

[28] Jan Parthey and Robert Baumgartl. 2004. Porting GCC to the TMS320-C6000 DSP Architecture. Proceedings of the Global Signal Processing Conference and Expo (GSPx), Santa Clara, CA, USA.

[29] Roberto Castañeda Lozano, Mats Carlsson, Frej Drejhammar, and Christian Schulte. 2012. Constraint-Based Register Allocation and Instruction Scheduling. In *Principles and Practice of Constraint Programming*. Springer Berlin Heidelberg, Berlin, Heidelberg, 750–766.

[30] Dick Grune, Kees Van Reeuwijk, Henri E Bal, Ceriel JH Jacobs, and Koen Langendoen. 2012. *Modern compiler design* (2 ed.). Springer Science & Business Media, New York, NY, USA.

[31] B. Ramakrishna Rau and Christopher D. Glaeser. 1981. Some scheduling techniques and an easily schedulable horizontal architecture for high performance scientific computing. In *Proceedings of the 14th annual workshop on Microprogramming, MICRO 1981, Chatham (Cape Cod), Massachusetts, USA*. IEEE/ACM, New York, NY, USA / Los Alamitos, CA, USA, 183–198. http://dl.acm.org/citation.cfm?id=802449

[32] George Hotz. 2017. openhexagon: An attempt at an open source toolchain for the Hexagon DSP. https://github.com/geohot/openhexagon.

[33] Lakshmi N. Chakrapani, John C. Gyllenhaal, Wen-mei W. Hwu, Scott A. Mahlke, Krishna V. Palem, and Rodric M. Rabbah. 2004. Trimaran: An Infrastructure for Research in Instruction-Level Parallelism. In *Languages and Compilers for High Performance Computing, 17th International Workshop, LCPC 2004, West Lafayette, IN, USA, September 22-24, 2004, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 3602)*. Springer, Berlin, Heidelberg, 32–41. doi:10.1007/11532378_4

[34] Pohua P Chang, Scott A Mahlke, William Y Chen, Nancy J Warter, and Wen-mei W Hwu. 1991. IMPACT: An architectural framework for multiple-instruction-issue processors. *ACM SIGARCH Computer Architecture News* 19, 3 (1991), 266–275.

[35] Vinod Kathail, Michael S. Schlansker, and B. Ramakrishna Rau. 2000. *HPL-PD Architecture Specification: Version 1.1*. Technical Report HPL-93-80 (R.1). Hewlett-Packard Laboratories. https://trimaran.org/docs/hpl-pd.pdf

[36] Björn Franke and Michael O'boyle. 2003. Array recovery and high-level transformations for DSP applications. *ACM Trans. Embed. Comput. Syst.* 2, 2 (May 2003), 132–162. doi:10.1145/643470.643472

[37] Alexei Kudriavtsev and Peter Kogge. 2005. Generation of permutations for SIMD processors. *SIGPLAN Not.* 40, 7 (June 2005), 147–156. doi:10.1145/1070891.1065931

[38] Douglas C. Montgomery. 2017. *Design and Analysis of Experiments* (9 ed.). John Wiley & Sons, Hoboken, NJ. Chapter 5: Introduction to Factorial Designs.

[39] Erven Rohou, François Bodin, André Seznec, Gwendal Le Fol, François Charot, and Frédéric Raimbault. 1996. *SALTO : System for Assembly-Language Transformation and Optimization.* Research Report RR-2980. INRIA. https://inria.hal.science/inria-00073718

[40] Jocher Glenn. 2021. YOLOv5 release v6.0. https://github.com/ultralytics/yolov5/releases/tag/v6.0

[41] Monica Lam. 1988. Software Pipelining: An Effective Scheduling Technique for VLIW Machines. In *Proceedings of the ACM SIGPLAN'88 Conference on Programming Language Design and Implementation (PLDI), Atlanta, Georgia, USA, June 22-24, 1988.* ACM, New York, NY, USA, 318–328. doi:10.1145/53990.54022

[42] Joseph A. Fisher. 1981. Trace Scheduling: A Technique for Global Microcode Compaction. *IEEE Trans. Computers* 30, 7 (1981), 478–490. doi:10.1109/TC.1981.1675827

[43] Wen-mei W. Hwu, Scott A. Mahlke, William Y. Chen, Pohua P. Chang, Nancy J. Warter, Roger A. Bringmann, Roland G. Ouellette, Richard E. Hank, Tokuzo Kiyohara, Grant E. Haab, John G. Holm, and Daniel M. Lavery. 1993. The superblock: An effective technique for VLIW and superscalar compilation. *J. Supercomput.* 7, 1-2 (1993), 229–248. doi:10.1007/BF01205185

[44] Scott A. Mahlke, David C. Lin, William Y. Chen, Richard E. Hank, and Roger A. Bringmann. 1992. Effective compiler support for predicated execution using the hyperblock. *SIGMICRO Newsl.* 23, 1–2 (Dec. 1992), 45–54. doi:10.1145/144965.144998

[45] Ziteng Yang, Jun Shirako, and Vivek Sarkar. 2024. Fully Verified Instruction Scheduling. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 299 (Oct. 2024), 26 pages. doi:10.1145/3689739

[46] Fabian Stuckmann, Moritz Weißbrich, and Guillermo Payá-Vayá. 2024. Energy-Aware Register Allocation for VLIW Processors. *J. Signal Process. Syst.* 96, 11 (Nov. 2024), 627–650. doi:10.1007/s11265-024-01930-x

[47] Xiaolei Zhao, Zhaoyun Chen, Yang Shi, Mei Wen, and Chunyun Zhang. 2023. Automatic End-to-End Joint Optimization for Kernel Compilation on DSPs. In *2023 60th ACM/IEEE Design Automation Conference (DAC).* IEEE, Piscataway, NJ, USA, 1–6. doi:10.1109/DAC56929.2023.10247901

A Per-Benchmark Performance Breakdown

Table 10 presents detailed performance data for all 26 benchmark instances (13 programs × 2 platforms) under Config F (full competition). We analyze three categories based on performance ratio (C_{expert}/C_{LASM}):

LASM excels (≥102%). Seven instances demonstrate LASM's advantages over expert assembly:

- `idamax` (+50.5%/+17.7%): LASM discovers aggressive pipelining opportunities that experts avoid due to complex predicated control flow. The benchmark's search pattern with conditional updates benefits from LAEMS's dependency relaxation.
- `prolongation` (+32.1%/+32.3%): LASM finds EC=7, II=12 configurations versus experts' conservative 4× loop unrolling, demonstrating systematic exploration benefits.
- `axpby` on MT3000-GPDSP (+13.8%): LAEMS successfully pipelines at EC=7 where manual optimization settles for smaller expansion factors.
- `fft_col/mul_coeff`: Nested loop structures benefit from LAEMS's parallel search across expansion counts.

Parity (±2%). Twelve instances achieve performance within 2% of expert assembly, indicating LASM matches expert optimization quality. These include compute-bound kernels (`gemm`, `scal`) where both approaches achieve near-optimal resource utilization.

Gap (<98%). Seven instances show performance below expert:

- `asum` (−15.3%/−9.5%): Experts achieve 6 vector instructions in a 6-cycle pipeline through careful manual reordering; LASM's height-based ordering utilizes only 5 vector instructions.
- `SpMV` (−8.1%/−13.9%): Post-scheduling register allocation cannot rearrange global blocks to reduce variable lifetimes, causing suboptimal register pressure management.

Table 10. Per-benchmark performance breakdown for Config F across 26 instances. C_{ideal}: theoretical minimum cycles; C_{expert}: expert assembly cycles; C_{LASM}: LASM cycles; Perf. Ratio: $C_{expert}/C_{LASM} \times 100\%$; $\mathcal{E}$: efficiency relative to theoretical optimum.

Platform	Program	C_{ideal}	C_{expert}	C_{LASM}	Perf. Ratio	$\mathcal{E}_{expert}$	$\mathcal{E}_{LASM}$	Analysis
Matrix DSP	gemm	3072	3108	3110	99.9%	98.8%	98.8%	Parity
Matrix DSP	gemvt	520	582	605	96.2%	89.3%	86.0%	Gap
Matrix DSP	asum	960	2056	2428	84.7%	46.7%	39.5%	Gap
Matrix DSP	axpby	768	1564	1566	99.9%	49.1%	49.0%	Parity
Matrix DSP	scal	480	1938	1938	100.0%	24.8%	24.8%	Parity
Matrix DSP	dot	480	1094	1075	101.8%	43.9%	44.7%	Parity
Matrix DSP	fft_col	5760	16006	15564	102.8%	36.0%	37.0%	LASM excels
Matrix DSP	fft_r2	400	1470	1498	98.1%	27.2%	26.7%	Parity
Matrix DSP	SpMV	729	2372	2581	91.9%	30.7%	28.2%	Gap
Matrix DSP	coeff_update	768	1451	1435	101.1%	52.9%	53.5%	Parity
Matrix DSP	mul_coeff	768	1857	1823	101.9%	41.4%	42.1%	Parity
Matrix DSP	prolongation	64	535	405	132.1%	12.0%	15.8%	LASM excels
Matrix DSP	idamax	768	6304	4188	150.5%	12.2%	18.3%	LASM excels
MT3000-GPDSP	gemm	3072	3100	3101	100.0%	99.1%	99.1%	Parity
MT3000-GPDSP	gemvt	684	749	773	96.9%	91.3%	88.5%	Gap
MT3000-GPDSP	asum	1280	3538	3909	90.5%	36.2%	32.7%	Gap
MT3000-GPDSP	axpby	1024	2066	1815	113.8%	49.6%	56.4%	LASM excels
MT3000-GPDSP	scal	640	1937	1936	100.1%	33.0%	33.1%	Parity
MT3000-GPDSP	dot	640	1060	1207	87.8%	60.4%	53.0%	Gap
MT3000-GPDSP	fft_col	7680	15845	15820	100.2%	48.5%	48.5%	Parity
MT3000-GPDSP	fft_r2	534	1295	1273	101.7%	41.2%	41.9%	Parity
MT3000-GPDSP	SpMV	972	3105	3605	86.1%	31.3%	27.0%	Gap
MT3000-GPDSP	coeff_update	1024	1594	1579	100.9%	64.2%	64.9%	Parity
MT3000-GPDSP	mul_coeff	1024	1597	1565	102.0%	64.1%	65.4%	LASM excels
MT3000-GPDSP	prolongation	86	537	406	132.3%	16.0%	21.2%	LASM excels
MT3000-GPDSP	idamax	1024	7359	6254	117.7%	13.9%	16.4%	LASM excels
Matrix DSP Summary				Geom. Mean: 103.6%		36.6%	37.9%	7/13 ≥100%
MT3000-GPDSP Summary				Geom. Mean: 101.6%		43.3%	44.0%	8/13 ≥100%

- `gemvt`/`dot`: Minor gaps stem from expert micro-optimizations in reduction epilogues.

These gaps highlight future optimization opportunities while demonstrating that LASM achieves competitive performance across diverse computational patterns.

B Custom Strategy Integration Guide

This appendix demonstrates how to extend LASM with a custom scheduling strategy, using Front-Back Block Fusion (FBBF) as an illustrative example. FBBF integrates through both scheduling interfaces (cyclic and linear). Adding a new strategy requires four localized modifications:

Step 1: Register Command-Line Option

Add a boolean flag or other custom options to the `Option` structure in `option.h`:

```
// option.h
struct Option {
    // ... existing options ...
    bool fbbfC = false; // FBBF for cyclic (loop) scheduling
    bool fbbfL = false; // FBBF for linear (basic block) scheduling
};
```

Add the corresponding command-line parsing logic in the `CommandParser(...)`:

```
// lasm.cc
bool CommandParser(int argc, const char* const* argv, iargs_t& _ias) {
    // ... existing code ...
    _ias.opts.fbbfC = CML.flag("--fbbfc"); // parse cyclic FBBF flag
    _ias.opts.fbbfL = CML.flag("--fbbfl"); // parse linear FBBF flag
    // ... existing code ...
```

```
    return true;
}
```

Step 2: Declare Strategy Interface

Declare strategy entry points in `strategy_library.h`. Strategies are organized by scheduling horizon: `loop` namespace for cyclic strategies and `base` namespace for linear strategies:

```
// strategy_library.h
namespace base { // linear scheduling strategies
    void FBBF(const BasicBlock&, const MetaManager&, const Option&, Schedules&);
};
namespace loop { // cyclic scheduling strategies
    void FBBF(const LoopBlock&, const MetaManager&, const Option&, Schedules&);
};
```

Step 3: Implement Strategy Wrapper

Implement the strategy wrapper in `strategy_library.cc`. The wrapper validates applicability, invokes the core algorithm, and submits results. The following pseudocode uses `loop::FBBF` as an example:

```
// strategy_library.cc
void loop::FBBF(const LoopBlock& context, const MetaManager& metas, const Option& opts, Schedules&
    ↪ candidates) {
    // validate applicability
    if (!isApplicable(context)) return;
    // invoke core algorithm
    auto beatBlocks = FBBF_Algorithm(context, metas, opts);
    // reconstruct loop with scheduled beats
    auto newLoop = createLoopBlock();
    newLoop->setName(context.name());
    newLoop->setIteration(context.iteration());
    for (auto& beat : beatBlocks) {
        newLoop->append(std::move(beat));
    }
    // package and submit candidate
    ScheduleMeta result(context);
    result.push(std::move(newLoop), metas); //save newLoop ->context
    result.finalize("LOOP:FBBF");
    candidates.append(std::move(result));
}
```

Step 4: Register with Scheduling Driver

Register the strategy in `codegen.cc` based on command-line options:

```
// codegen.cc
bool codegen(RootBlock& context, MetaManager& metas, const Option& opts, ...) {
    CyclicStrategies cyclicBox; // loop strategy container
    LinearStrategies linearBox; // basic block strategy container
    // register FBBF based on options
    if (opts.fbbfC) {
        cyclicBox.emplace_back(&loop::FBBF);
    }
    if (opts.fbbfL) {
        linearBox.emplace_back(&base::FBBF);
    }
    // ... register other strategies ...
    // run multi-strategy competitive scheduling
    Driver scheduler(opts, std::move(cyclicBox), std::move(linearBox));
    if (!scheduler.run(context, metas)) {
        return false;
    }
    // post-scheduling:
    // register allocation (handle remaining variables), code emission
    // ...
    return true;
}
```

Commands and Usage Examples

The general command-line syntax for LASM is:

```
$ lasm <file ...> [options] --layout <file> --output <file...>
```

- `lasm`: The LASM executable.
- `<file ...>`: One or more input LAN program files.
- `[options]`: Optional switches to enable specific scheduling strategies (see examples below).
- `-layout <file>`: Specifies the user-provided memory layout file (JSON format).
- `-output <file...>`: Writes the output to the specified file(s).

We take the LAN program example in Figure 2, `testing_axpby.lan`, as input:

Basic usage (FBBF on basic blocks only):

```
$ lasm testing_axpby.lan --fbbfl --layout m3_memlayout.json --output a.s
```

This command applies Front-Back Block Fusion (FBBF) to basic blocks only. Loop regions not matched by any cyclic strategy fall back to NOP-based delay slot filling.

FBBF on both loop and basic blocks:

```
$ lasm testing_axpby.lan --fbbfc --fbbfl --layout m3_memlayout.json --output a.s
```

This enables FBBF for both loop blocks (`-fbbfc`) and basic blocks (`-fbbfl`).

Full multi-strategy competition:

```
$ lasm testing_axpby.lan --fbbfc --fbbfl --laems --ims \
      --tdlsc --tdlsl --layout m3_memlayout.json --output a.s
```

Received 10 November 2025; Revised 26 January 2026; Accepted 14 April 2026; Online AM 23 April 2026